

Manuel Babet

Table des matières

Babet — Manuel utilisateur	5
Pour démarrer	5
Modules	5
Cookbook	5
Génération du PDF	5
Conventions utilisées dans la doc	6
Méthodologie de développement	6
Pour démarrer	6
Téléchargement	6
Compilation depuis les sources	6
Trois manières de lancer un script Lua	6
1. Mode dossier	6
2. Mode embarqué (binaire autonome)	7
3. Embarqué via PATH (dossier + auto-détection)	7
Invocation en ligne de commande	7
Premier script	7
Pour aller plus loin	8
Modèle de sécurité	8
Ce contre quoi le durcissement <i>protège</i>	8
Ce contre quoi il <i>ne protège pas</i>	8
Pattern recommandé : moindre privilège	9
argparse — parseur d’arguments CLI (module Lua)	9
Pourquoi	9
API	9
Champs de la table <code>opts</code>	9
Contrat de retour de <code>parse(src?)</code>	10
Source des arguments	10
Formes d’arguments acceptées	10
Exemple rapide	10
Contrat d’erreur	11
Décisions de design	12
Hors v1	12
babet.exec — exécuter un programme externe	12
Pourquoi	12
API	12
Exemples rapides	13
Contrat d’erreur	14
Décisions de design	14
Hors v1	14
babet fs — système de fichiers	15

Pourquoi	15
API — Existence, type, attributs	15
API — Création, suppression, déplacement	15
mkdir en détail	16
API — Manipulation de chemins	16
API — Listing et recherche	16
API — Copie	17
API — Hashes et checksums	17
API — Checksums en mémoire	17
Exemples rapides	17
Contrat d’erreur	18
Décisions de design	18
Hors v1	19
babet.http — client HTTP	19
Pourquoi	19
API	19
Table opts	19
Table response	20
Exemples rapides	20
Contrat d’erreur	20
TLS / trust store	20
Décisions de design	21
Hors v1	21
babet.inotify — surveillance d’événements fichier	21
Pourquoi	21
API	21
Argument events	22
Événement renvoyé par read	22
Exemple rapide	22
Contrat d’erreur	23
Décisions de design	23
Hors v1	23
babet.json — encode/décode JSON	23
Pourquoi	24
API	24
Exemple rapide	24
null, empty_array, as_array	24
Contrat d’erreur	25
Décisions de design	25
Hors v1	25
logging — logger avec niveaux (module Lua)	25
Pourquoi	25
API	25
Exemple rapide	26
Contrat d’erreur	26
Décisions de design	26
Hors v1	26
babet.signal — signaux POSIX	26
Pourquoi	26
API	27
Exemple rapide	27
Comment les callbacks s’exécutent	27

Contrat d'erreur	28
Décisions de design	28
Hors v1	28
babot.socket — sockets TCP client et serveur	28
Pourquoi	29
API	29
Constructeurs	29
Méthodes (socket client)	29
Méthodes (socket serveur)	29
Exemples rapides	30
Client TCP echo	30
Serveur line-protocol simple avec arrêt propre	30
Contrat d'erreur	30
Décisions de design	31
Hors v1	31
babot.sqlite — base de données SQL embarquée	31
Pourquoi	31
API	31
Ouverture et fermeture	31
Exécution et requêtes	32
Mapping de types	32
Exemples rapides	32
Contrat d'erreur	33
Décisions de design	33
Hors v1	33
babot.strings — manipulation de chaînes	34
Pourquoi	34
API	34
Exemple rapide	34
Contrat d'erreur	34
Décisions de design	34
Hors v1	35
babot.sys — utilitaires système	35
Pourquoi	35
API	35
Constantes runtime	35
Exemple rapide	36
Contrat d'erreur	36
Décisions de design	36
Hors v1	37
babot.tables — helpers de manipulation de tables	37
Pourquoi	37
API	37
Exemple rapide	38
Contrat d'erreur	38
Décisions de design	38
Hors v1	38
babot.time — horloges monotone et temps réel	38
Pourquoi	39
API	39
API — utilitaires de dates et durées (v1.8.0+)	39

Exemple rapide	40
Contrat d'erreur	40
Décisions de design	40
Hors v1	40
babet.socket TLS — connect_tls et starttls	41
Pourquoi	41
API	41
Exemples rapides	41
Connexion à IRC over TLS	41
Upgrade STARTTLS (soumission SMTP)	42
Serveur auto-signé (dev / CA privée)	42
Skip de la vérification (TESTS UNIQUEMENT)	42
Contrat d'erreur	42
Trust store	42
Décisions de design	43
Hors v1	43
babet.toml — parseur TOML	43
Pourquoi	43
API	43
Exemple rapide	44
Contrat d'erreur	44
Décisions de design	44
Hors v1	44
babet.user	44
Pourquoi	44
API	44
Table renvoyée en cas de succès	45
Exemple rapide	45
Contrat d'erreur	45
Décisions de design	45
Hors v1	46
babet.workers — jobs parallèles	46
Pourquoi	46
API	46
Argument code	47
Argument args	47
Exemples rapides	47
Fetches HTTP parallèles	47
Travail CPU-bound avec cancellation	47
Pattern pool de workers	48
Contrat d'erreur	48
Partage de données	49
Décisions de design	49
Hors v1	49
Cookbook	49
Surveiller un dossier et envoyer les nouveaux fichiers par email	49
S'assurer qu'un user système existe, puis basculer dessus	50
Récupérer beaucoup d'URLs en parallèle	50
Arrêt propre d'un script long-running	51
Lire une config TOML, valider, persister en SQLite	51
Cache avec TTL lisible, échéances en timestamps ISO	52
Pretty-print d'une valeur Lua	52

Babet — Manuel utilisateur

Babet, n.m. — mot régional du sud-est de la France (Lyonnais, Forez, Dauphiné, Savoie, ainsi que la Suisse romande voisine) désignant une pomme de pin. Petit, léger, plein de graines, et capable d’allumer un feu — comme ce binaire.

Babet est un binaire Lua 5.5 standalone pour le scripting et l’automatisation sous Linux. Ceci est le manuel de référence, organisé en un fichier par module pour que chaque section reste ciblée et facile à maintenir.

Pour démarrer

- [getting-started.md](#) — installation, premier script, les trois modes de lancement (dossier / embarqué / via PATH).
- [security.md](#) — ce contre quoi le durcissement protège et ne protège pas, patterns de moindre privilège.

Modules

Chaque module vit dans son propre fichier sous `modules/` :

Module	Rôle
argparse	Parseur d’arguments de ligne de commande.
exec	Exécuter des programmes externes et capturer la sortie.
fs	Système de fichiers : listing, copie, hash, attrs.
http	Client HTTP (basé sur <code>cpp-httpplib</code>).
inotify	Surveillance d’événements fichier (<code>inotify(7)</code>).
json	Encode/décode JSON (<code>nlohmann/json</code>).
logging	Logger avec niveaux.
signal	Signaux POSIX : <code>SIGTERM</code> , <code>SIGINT</code> , ...
socket	TCP client/serveur avec timeouts.
sqlite	Base de données SQL embarquée.
strings	Manipulation de chaînes (<code>split</code> , etc.).
sys	<code>which</code> , <code>hostname</code> , <code>env</code> , <code>pid</code> , ...
tables	Manipulation de tables (<code>mergeTables</code> , <code>deepCopyTable</code>).
time	Horloges monotone et temps réel.
tls	Sockets TLS : <code>connect_tls</code> , <code>starttls</code> .
toml	Parseur TOML (<code>tomlplusplus</code>).
user	Lookups d’utilisateurs système via NSS.
workers	Jobs parallèles (vrais threads OS).

Le module [user](#) est la référence canonique du pattern de documentation : *Pourquoi*, *API*, *Exemple rapide*, *Contrat d’erreur*, *Décisions de design*, *Hors v1*. Les autres modules varient dans le respect strict de ce template ; l’objectif au fil du temps est de les aligner.

Cookbook

[cookbook.md](#) — recettes concrètes : watch de dossier + email, fetches HTTP parallèles, arrêt propre, TOML+SQLite, etc.

Génération du PDF

Le manuel peut être exporté en un seul PDF :

```
cd docs
./build_doc.sh          # utilise pandoc + LaTeX
```

Voir docs/manual_order_fr.txt pour l'ordre des fichiers ; c'est le seul endroit à éditer quand on ajoute ou réordonne des chapitres.

Conventions utilisées dans la doc

- **Tables d'API** : chaque module commence par une petite table qui liste les fonctions, leurs signatures, et ce qu'elles renvoient.
- **Contrat d'erreur** : une section dédiée explique quelles erreurs sont levées (`luaL_error`) et lesquelles sont renvoyées en (`nil`, `err`). Cohérent entre tous les modules.
- **Décisions de design** : quand un choix n'est pas évident ("pourquoi pas récursif ?", "pourquoi obligatoire ?"), il y a un bref rationnel.
- **Hors v1** : une courte liste de ce qui pourrait être ajouté de manière additive plus tard sans casser le SemVer.

Méthodologie de développement

Babet est développé par un seul auteur avec une assistance IA substantielle (principalement Claude / Anthropic, avec des audits croisés de ChatGPT, Gemini et Mistral). Chaque suggestion de ces outils est traitée comme une hypothèse à vérifier — les audits croisés par IA identifient régulièrement des « bugs » qui n'existent pas, ou proposent des changements d'API qui n'ont pas changé. Chaque modification est validée par la batterie de 890 tests avant chaque release.

Pour démarrer

Téléchargement

Des binaires précompilés pour x86_64, aarch64 (RPi4) et armv6l (RPi0) sont disponibles sur la [page des releases](#).

Compilation depuis les sources

Babet embarque toutes ses dépendances (Lua, OpenSSL, SQLite, miniz, nlohmann/json, cpp-httpplib, tomlplusplus), donc les seuls prérequis sur ton système sont :

- un compilateur C++23 (GCC ou Clang récent)
- CMake (3.20)
- wget et unzip

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh           # télécharge les deps et compile
                           # (~5 min au premier lancement, plus vite après)
./run_tests.sh             # harness offline – doit afficher 890 PASS / 0 FAIL
```

Le script de build télécharge chaque dépendance depuis sa source upstream, vérifie son SHA256, puis compile statiquement. Si la source upstream est temporairement indisponible ([lua.org](#) a notamment connu des pannes), le script bascule sur la Wayback Machine d'Internet Archive — le check SHA256 reste appliqué.

Le binaire produit est dans `test/babet`.

Trois manières de lancer un script Lua

Babet supporte trois modes de lancement :

1. Mode dossier

Pointe le binaire vers un dossier contenant `main.lua` :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

`require("X")` dans `main.lua` cherche `X.lua` dans le même dossier.

2. Mode embarqué (binaire autonome)

Empaquète un script et ses modules dans un binaire autonome :

```
./test/babet --create-exe . myapp
./myapp      # lance main.lua depuis le ZIP embarqué
```

En interne, Babet append un ZIP à son propre binaire et lit `main.lua` (plus tous les modules `require`) depuis ce ZIP au runtime. Le `myapp` résultant est totalement autonome — pas besoin d’avoir Babet installé sur la machine cible, juste glibc.

3. Embarqué via PATH (dossier + auto-détection)

Si Babet est sur `$PATH` et qu’on fait `chmod +x main.lua` après avoir ajouté une ligne shebang :

```
#!/usr/bin/env babet
print("hello")

chmod +x main.lua
./main.lua
```

Babet utilise le dossier du script comme dossier de travail, donc `require("helpers")` trouvera `helpers.lua` à côté de `main.lua`.

Invocation en ligne de commande

En plus des trois modes de lancement ci-dessus, Babet accepte quelques flags standards :

Flag	Effet
-h, --help	Affiche l’aide et sort avec code 0.
-V, --version	Affiche <code>babet <version></code> et sort avec code 0.
-c <dir> <out>, --create-exe <dir> <out>	Crée un exécutable autonome nommé <out> en embarquant <dir> (qui doit contenir <code>main.lua</code>).

Tout autre argument commençant par - est traité comme une **option inconnue** : Babet affiche `Unknown option: ...` + un indice pour utiliser `--help`, et sort en code 1 plutôt que d’essayer de l’interpréter comme un nom de dossier. Les dossiers dont le nom commence légitimement par - peuvent toujours être passés via `./-dirname` (convention POSIX).

```
babet --version    # babet 2.0.2
babet --help      # usage complet
babet --bogus     # Unknown option: --bogus
                  # Try 'babet --help' for more information.
```

La même version est aussi exposée aux scripts via `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` en integer) — voir [sys](#).

Premier script

Une fois le binaire compilé, essaie ceci :

```
-- main.lua
print("Babet dit bonjour")
print("PID :", babet.pid())
print("Hôte :", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status :", r.status)
else
```

```
print("Échec HTTP :", err)
end
```

Lance-le avec `./test/babet .` (ou un autre mode au choix).

Pour aller plus loin

- Chaque module a sa propre page sous `modules/`.
- Le module `user` est un petit exemple complet du pattern de documentation utilisé partout — lis-le comme la référence canonique.
- Pour les utilitaires de dates et durées (timestamps ISO 8601, durées lisibles comme "5m" ou "2h30m"), voir `time` — la sous-table `babet.time.*` couvre le parsing et le formatage.
- Vois `security.md` avant d'exposer des scripts Babet à quoi que ce soit qui pourrait recevoir de l'input non fiable.

Modèle de sécurité

Babet exécute des scripts Lua de confiance. Ce n'est pas une sandbox.

Les scripts s'exécutent avec la bibliothèque standard Lua complète (`luaL_openlibs`), et Babet expose en plus des primitives système comme `exec`, `remove`, `rmdirAll` et `chdir`. Un script a donc les mêmes privilèges que le processus qui le lance : il peut lire, modifier ou supprimer des fichiers, et lancer des commandes arbitraires. C'est volontaire — comme `make`, un script shell, ou n'importe quel outil de build, Babet est conçu pour exécuter du code que tu contrôles.

Ne lance que des `main.lua` et des modules `require` auxquels tu fais confiance. **N'utilise pas** Babet pour exécuter du Lua venant de sources non fiables ; il n'offre aucune isolation contre du code hostile, et n'est pas pensé pour. Si tu dois exécuter des scripts non fiables, utilise plutôt une sandbox Lua dédiée à cet usage.

Ce contre quoi le durcissement *protège*

Le travail de durcissement dans Babet protège les usages *légitimes* contre les accidents et les attaques de supply chain :

- **Gestion des chemins / symlinks** utilise `lexically_relative()` et des guards `is_within()` composant par composant (jamais des checks de préfixe de string), donc un `cp src dst/` ne peut pas s'évader dans un arbre voisin via un composant symlinké.
- **Cleanup des groupes de processus** dans `babet.exec` garantit que les enfants forkés ne survivent pas au parent et ne laissent pas de processus zombies quand le script est interrompu.
- **Limites de sortie** sur `babet.exec` bornent la quantité de stdout et stderr capturés en mémoire (par défaut 16 MiB chacun, configurable), pour qu'un sous-processus emballé ne puisse pas OOM le processus Babet.
- **Checksums des dépendances** : chaque dépendance embarquée est SHA256-pinnée dans `build_local.sh`. Un hash vide refuse la build. Un mismatch supprime le fichier téléchargé et sort en erreur. Ça protège contre un upstream compromis ou une archive Wayback Machine altérée (source de fallback).
- **Vérification TLS** est activée par défaut (`verify=true`). La désactiver demande un `verify=false` explicite par appel — pas de fallback silencieux. Des CA stores personnalisés peuvent être passés via `ca_cert` ou `ca_path`, ou via les variables d'environnement `SSL_CERT_FILE` / `SSL_CERT_DIR`.

Ce contre quoi il *ne protège pas*

- Un script malveillant. Babet n'a pas de sandbox. Si tu fais `exec` sur de l'input utilisateur, tu as une injection shell. Si tu fais `loadstring` sur de l'input utilisateur, tu as une exécution de code arbitraire.
- Un attaquant déterminé qui a déjà obtenu une exécution de code sur la machine qui fait tourner Babet. Le durcissement rend les erreurs accidentelles visibles, pas les attaques adversariales impossibles.
- Les problèmes OS-level (exploits kernel, évasions de conteneur, escalade de privilèges). Babet est un binaire userland ordinaire.

Pattern recommandé : moindre privilège

Quand tu fais tourner Babet en service, traite-le comme n'importe quel processus non privilégié :

```
# En root, crée un user système dédié, sans shell.
```

```
useradd --system \
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp
```

```
# Utilise babet.user.exists("myapp") dans ton script d'install
# pour vérifier que le user est provisionné avant de lancer le
# daemon.
```

Combine ça avec des options de unit systemd comme `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, et `CapabilityBoundingSet=` (vide sauf si tu as vraiment besoin d'une capability). Le kernel fait bien plus que Babet ne peut le faire pour contenir un script qui se comporte mal ; laisse-le faire.

argparse — parseur d'arguments CLI (module Lua)

Un parseur d'arguments déclaratif pour les scripts en ligne de commande. Embarqué en module Lua pur, chargé via `require("argparse")`. Renvoie ses résultats sous forme de données (pas via des `print` ou `os.exit`) — le script décide quoi en faire.

Pourquoi

Un script Babet qui prend des arguments ne devrait pas avoir à réinventer le parsing (positionnel vs flag, défauts, texte d'aide, coercion de type, choix) à chaque fois. `argparse` rend les patterns CLI courants accessibles en une ligne — et reste hors du chemin du script en renvoyant tout sous forme de données : l'aide est un résultat, pas un effet de bord ; les erreurs sont une valeur de retour, pas une exception.

API

```
local argparse = require("argparse")
local p = argparse(prog, description?)
```

Méthode du builder	Renvoie	Usage
<code>p:flag(spec, opts?)</code>	<code>p</code> (chaînable)	Flag booléen (sans valeur).
<code>p:option(spec, opts?)</code>	<code>p</code> (chaînable)	Flag qui prend une valeur.
<code>p:argument(name, opts?)</code>	<code>p</code> (chaînable)	Argument positionnel.
<code>p:parse(src?)</code>	(<code>res</code> , <code>err</code>)	Parse les arguments (voir ci-dessous).
<code>p:get_usage()</code>	<code>string</code>	Génère le texte d'aide manuellement.

`spec` pour `flag` et `option` est **une seule string** contenant un ou plusieurs noms séparés par des espaces : `"-v"`, `"--verbose"`, ou `"-v --verbose"`. Les noms doivent commencer par `-`.

`name` pour `argument` est un identifiant simple (sans tiret initial).

Champs de la table `opts`

Champ	Type	Effet
help	string	Texte d'aide affiché par <code>get_usage()</code> .
default	any	Valeur renvoyée quand le flag/option/argument est absent.
dest	string	Force la clé de destination dans <code>res</code> . Défaut : 1er nom long sans tirets, sinon 1er nom court sans tiret.
required	boolean	Marque comme requis. Pour les positionnels, défaut à <code>true</code> sauf si <code>default</code> est défini.
choices	array de strings	Restreint la valeur brute à un ensemble fini.
convert	fonction <code>string -> any</code>	Coerce la string brute. Renvoyer <code>nil</code> est traité comme un échec (la fonction <code>convert</code> peut renvoyer <code>(nil, "raison")</code>).

Contrat de retour de `parse(src?)`

- **Succès** : `(res, nil)` — `res` est une table indexée par `dest`, contenant la valeur parsée (ou défaut) pour chaque flag / option / argument déclaré.
- **Aide demandée** : `({ help = true, usage = "<texte>" }, nil)` quand l'utilisateur a passé `-h` ou `--help`. L'aide est un **succès**, pas une erreur. Le script décide quoi faire (typiquement afficher `res.usage` et sortir).
- **Échec** : `(nil, err)` — string lisible pour tout problème d'entrée utilisateur (option inconnue, valeur manquante, choix invalide, erreur de conversion, argument requis manquant, positionnel en trop).

`parse()` **ne lève jamais** sur entrée utilisateur — toute erreur utilisateur passe par `(nil, err)`. Seul un **mauvais usage du builder** (spec vide, nom sans -, nom dupliqué, etc.) lève via `error()`, parce que c'est un bug du programmeur.

Source des arguments

- `p:parse()` (sans argument) lit la table globale `arg`, indices `1..n` (on s'arrête au premier trou). Les indices `<= 0` (chemin du binaire, dossier) sont ignorés — ce sont les slots internes de Babet, pas des arguments utilisateur.
- `p:parse({ "a", "b" })` parse un array explicite. Utile pour les tests ou pour parser des arguments qui ne viennent pas du shell.

Formes d'arguments acceptées

- `--long val`
- `--long=val`
- `-s val`
- `-s=val`
- `--` termine le parsing des options — tout token suivant est traité comme un positionnel, même s'il commence par `-`.

Les options courtes agglomérées (`-abc`) et la forme `-fvalue` (option courte immédiatement suivie de sa valeur, sans séparateur) **ne sont pas supportées** en v1.

Exemple rapide

```
local argparse = require("argparse")

local p = argparse("myapp", "Traite des fichiers.")
:flag("-v --verbose", { help = "Sortie verbeuse" })
:option("-o --output",
```

```

        { help = "Fichier de sortie", default = "out.txt" })
:option("-n --count",
        { help = "Combien", convert = tonumber, default = 1 })
:argument("input", { help = "Fichier d'entrée" })

local args, err = p:parse()

-- L'aide est un succès, pas une erreur.
if args and args.help then
    print(args.usage)
    return
end

if err then
    io.stderr:write("erreur : ", err, "\n")
    io.stderr:write(p:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)

```

À l'exécution :

```
$ ./myapp.lua data.csv --count 5 -v
data.csv  out.txt  5  true
```

```
$ ./myapp.lua --help
Usage: myapp [options] <input>
```

Traite des fichiers.

Arguments:

input Fichier d'entrée

Options:

```

-h, --help          show this help
-v, --verbose       Sortie verbeuse
-o, --output <value> Fichier de sortie
-n, --count <value> Combien

```

Contrat d'erreur

- **Mauvais usage du builder** (`p:flag("")`, `p:option("foo")` sans -, nom dupliqué, `p:argument("-bad")`) → lève via `error()`. Ce sont des bugs du programmeur.
- **`p:parse(...)`** ne lève jamais sur entrée utilisateur. Tous les problèmes d'entrée utilisateur sont rapportés en (`nil`, "message") :
 - "unknown option '--xxx'"
 - "option '--xxx' requires a value"
 - "flag '-v' does not take a value"
 - "missing required option '--xxx'"
 - "missing required argument 'name'"
 - "argument 'name': invalid choice 'xxx'"
 - "option '--xxx': <message du convert>"
 - "unexpected argument 'xxx'"
- Une fonction `convert` qui lève elle-même est **interceptée** par `parse()` et transformée en (`nil`, "<...>: conversion error"). Le script ne voit jamais d'exception sortir de `parse()`.

Décisions de design

- **L'aide est un succès, pas un effet de bord.** Renvoyer `{ help = true, usage = "..."` plutôt que d'afficher et de sortir laisse le script décider : afficher sur stdout, logger dans un fichier, envoyer à une GUI, préfixer une bannière, peu importe. La bibliothèque ne décide pas à ta place, cohérent avec le reste de Babet où aucun module ne sort unilatéralement du process.
- **(res, err) partout sur entrée utilisateur, error() seulement sur bug du builder.** Même convention que le reste de Babet (`user`, `json`, `sqlite`, etc.).
- **Spec en une seule string** ("`-v --verbose`"), pas une chaîne d'appels. Découvrable, concis, et évite l'ambiguïté quand plusieurs noms sont déclarés ensemble.
- **choices est testé sur la string brute, puis convert appliqué.** Si tu combines les deux, exprime `choices` en strings — ex. `{ choices = {"1","2","3"}, convert = tonumber }`. Piège documenté, mais conscient : faire l'inverse forcerait les valeurs de `choices` à être du type converti, ce qui couplerait deux features indépendantes.
- **Un positionnel avec default est implicitement optionnel.** Évite le boilerplate `required = false` dans le cas courant.
- **parse() lit arg[1..n]** — jamais `arg[0]` ni les indices négatifs, où Babet place ses propres slots. Juste ce que l'utilisateur a réellement tapé.

Hors v1

Additif — pourrait être ajouté plus tard sans casser le SemVer :

- Sous-commandes (style `git commit` / `git push`).
- Options courtes agglomérées (`-abc` pour `-a -b -c`).
- Options courtes avec valeur attachée (`-fvalue`).
- Positionnels variadiques (`nargs = "+"` ou `"*"`).
- Groupes d'arguments (regroupement visuel dans l'aide).

babet.exec — exécuter un programme externe

Lance un sous-processus, capture son stdout / stderr, et récupère son code de sortie sous forme de données structurées. Là où `os.execute` te donne juste un entier, `babet.exec` te donne une table de résultat avec tout ce dont tu as réellement besoin.

Pourquoi

Presque chaque script non trivial finit par shell-out — vers `git`, `useradd`, `convert`, `ffmpeg`, peu importe. `os.execute` est trop rudimentaire (pas de sortie capturée, dangers de quoting shell), et `io.popen` ne te laisse pas à la fois écrire dans stdin et lire stdout proprement, n'expose pas le code de sortie sans parsing, et n'a pas de timeout.

`babet.exec` est l'alternative structurée : `argv` passé en liste (pas de shell, pas de bugs de quoting), sortie capturée en mémoire avec cap borné, timeout optionnel, stdout/stderr séparés.

API

```
result, err = babet.exec(cmd, args?, opts?)
```

Argument Type		Notes
cmd	string	Programme à exécuter. PATH est cherché.
args	table (optionnel)	Array d'arguments strings — chacun devient un élément argv séparé, pas de parsing shell.
opts	table (optionnel)	Voir ci-dessous.

Champs de `opts` :

Champ	Type	Défaut	Notes
cwd	string	hérité	Dossier de travail pour le child.
env	table {KEY = value, ...}	hérité	Fusionné avec l'environnement parent, jamais remplacé. Override : passe la clé. Vider : passe "" (le child la voit vide). Omettre une clé = hériter ; il n'y a pas d'unset en v1.
stdin	string	""	Données envoyées au stdin du child.
timeout	number (s)	aucun	Au dépassement : SIGTERM au groupe de processus entier (arrêt propre possible), 2 s de grâce, puis SIGKILL .
max_output	integer (octets)	10 Mio	Cap dur par flux sur la sortie capturée (plafond : 2 Gio). L'excès est silencieusement perdu et signalé via *_truncated.

Table de résultat en cas de lancement réussi :

```
{
  stdout = "...",          -- stdout capturé, jusqu'à max_output octets
  stderr = "...",          -- stderr capturé, jusqu'à max_output octets
  code = 0,                 -- code de sortie (0..255)
  timed_out = false,        -- true si tué par le timeout
  stdout_truncated = false, -- true si plus de max_output a été produit
  stderr_truncated = false,
}
```

Exemples rapides

```
-- Capture basique
local r = assert(babet.exec("git", { "rev-parse", "HEAD" }))
if r.code == 0 then
  print("HEAD :", r.stdout:gsub("\n$", ""))
end

-- Code non-zéro n'est PAS une erreur
local r = assert(babet.exec("grep", { "needle", "/etc/passwd" }))
if r.code == 0 then
  print("trouvé :", r.stdout)
elseif r.code == 1 then
  print("pas trouvé")
else
  print("grep a échoué :", r.stderr)
end

-- Pipe des données dans stdin
local r = assert(babet.exec("sha256sum", { "-" }, {
  stdin = "hello world\n",
}))
print(r.stdout)
-- a948904f2f0f479b8f8197694b30184b0d2ed1c1cd2a1ec0fb85d299a192a447 -

-- Augmenter env (sans remplacer)
local r = assert(babet.exec("env", nil, {
  env = { MY_FLAG = "yes" },
}))
-- Le child voit MY_FLAG=yes plus tout ce que le parent avait déjà.

-- Timeout + troncature
local r = assert(babet.exec("yes", nil, {
  timeout = 0.5,
```

```

    max_output = 1024,
  ))
print(r.timed_out, r.stdout_truncated) -- true  true

-- cwd différent
local r = assert(babet.exec("ls", nil, { cwd = "/var/log" }))

```

Contrat d'erreur

- **(nil, err)** seulement si le lancement lui-même échoue : programme introuvable, cwd n'existe pas, fork raté, OOM, etc.
- **(result, nil)** pour tous les autres cas, y compris :
 - Le programme a tourné et sorti non-zéro → `result.code > 0`.
 - Le programme a été tué par le timeout → `result.timed_out` vaut `true`, `result.code` reflète la sortie par signal.
 - Le programme a produit plus de sortie que `max_output` → le flag `*_truncated` concerné vaut `true`, les données capturées s'arrêtent au cap.
- **Validation de opts** : types invalides (`opts.cwd` pas une string, `opts.env` pas une table, clés de `opts.env` avec = ou NUL, `opts.timeout` 0 ou NaN/Inf, `opts.max_output` non entier ou > 2 GiB) → lève via `luaL_error`. Ce sont des bugs côté appelant.

Décisions de design

- **Code de sortie erreur.** Le bug le plus courant avec `os.execute` / les wrappers `exec` est de traiter une sortie non-zéro comme une exception. Beaucoup de programmes utilisent les codes de sortie pour des états significatifs (`grep` renvoie 1 quand pas de match, `diff` renvoie 1 quand les fichiers diffèrent, etc.). L'appelant décide de ce qui compte comme échec.
- **Pas de shell.** `args` est un array, chaque élément est un slot argv, pas besoin de quoting et pas de surprises (`$(rm -rf $HOME)`). Si tu veux vraiment un shell, passe "`sh`"
 - { "-c", "ta commande" }.
- **env fusionne avec le parent.** Remplacer l'environnement complet est rarement ce qu'on veut et casse généralement les programmes qui s'attendent à `PATH` ou `LANG`. Pour override une variable, passe-la. Pour la vider, passe-la en string vide et le child la verra vide (POSIX n'a pas de sémantique "unset" au moment du `exec` sans effort supplémentaire).
- **SIGTERM d'abord, SIGKILL ensuite.** Au timeout, le groupe de processus entier (le child crée le sien via `setpgid`) reçoit `SIGTERM` — les programmes propres peuvent flusher et sortir — puis `SIGKILL` après 2 s de grâce. `kill(-pid)` couvre aussi les descendants (un `sh -c` qui a forké, par exemple).
- **max_output est par flux, pas total.** 10 Mio stdout + 10 Mio stderr est le cap. Assez grand pour capturer la plupart des logs de build, assez petit pour qu'un sous-process emballé ne puisse pas OOM le process Babet.
- **Cap dur de 2 GiB sur max_output.** À peu près tout ce qui a besoin de plus de 2 GiB de stdout capturé devrait rediriger vers un fichier via un wrapper shell, p. ex. `exec("sh", { "-c", "ta-cmd > /tmp/log" })`.
- **stdin est une string, pas un callback.** Streamer des gigaoctets dans un child est rare et hors scope ; le cas courant est "envoie cette petite entrée et lis la réponse".

Hors v1

- **Interruption par babet.signal pendant l'exécution.** `exec` n'est pas signal-aware : un signal géré reçu pendant qu'un child tourne ne rend pas (`nil`, "`interrupted`") — le callback est dispatché après le retour d'`exec`. Utilise `opts.timeout` pour borner les exécutions longues.
- Streaming stdout / stderr (callback ligne-par-ligne pendant l'exécution). Actuellement toute la capture est renvoyée d'un coup.
- Streaming stdin (callback qui produit des chunks d'input).
- Redirection directe par fd (`stdout = "/tmp/log"`). Facile à ajouter plus tard si besoin ; le workaround par wrapper shell ci-dessus couvre la plupart des cas.
- Gestion de groupe de process au-delà du child. L'implémentation actuelle teardown correctement le groupe de process du child sur timeout / interrupt — voir [security](#).

babét fs — système de fichiers

Un ensemble plat d'opérations système de fichiers exposées directement sur **babét** (pas de sous-namespace, précède la convention). Couvre les besoins quotidiens que `os.*` et `io.*` de Lua ne gèrent pas : listing récursif, copie avec attributs, hashing, manipulation de chemins, attributs, liens.

Pourquoi

`io` et `os` de Lua te donnent `open`, `rename`, `remove`, et c'est à peu près tout. Tout ce qui va au-delà — lister un dossier, copie récursive, calculer un hash, interroger `mtime` — demande des programmes externes ou du FFI bas niveau. **babét** rassemble ça dans un set unique de fonctions qui se comportent de manière cohérente (chemins en string, erreurs en `(nil, "...")`).

Tous les chemins sont acceptés en string. L'expansion du tilde (`~/x`) n'est **pas** automatique — l'appelant doit résoudre `$HOME` lui-même si besoin. Les fonctions n'expansent jamais les globs ; pour le support glob, utilise **babét.find** avec un pattern explicite.

API — Existence, type, attributs

Fonction	Renvoie
<code>babét.fileExists(path)</code>	boolean
<code>babét.isFile(path)</code>	boolean (true seulement pour fichiers réguliers)
<code>babét.isDir(path)</code>	boolean
<code>babét.fileSize(path)</code>	integer octets (nil, err)
<code>babét.getAttributes(path)</code>	table avec <code>mtime</code> , <code>mode</code> , <code>size</code> , <code>uid</code> , <code>gid</code> , <code>inode</code> , etc. (nil, err)
<code>babét.setAttributes(path, uid, gid, mode?)</code>	(true, nil) (nil, err) — set owner/group, optionnellement le mode
<code>babét.symlinkAttr(path, uid, gid)</code>	(true, nil) (nil, err) — change owner/group du lien lui-même (<code>lchown</code> , ne suit pas le symlink) ; pendant de <code>setAttributes</code>
<code>babét.getMode(path)</code>	integer octal (nil, err)
<code>babét.setMode(path, mode)</code>	(true, nil) (nil, err) — mode est un integer (utilise <code>0x1ed</code> pour <code>0755</code> , ou construis-le avec <code>tonumber("755", 8)</code>)

Canonique : camelCase pour les composés Babet (`isFile`, `isDir`, `symlinkAttr`), minuscules pour les noms hérités de POSIX (`mkdir`, `chdir`, `touch...`). Les anciens noms `isfile`, `isdir` et `symlinkattr` restent des alias **dépréciés** des mêmes fonctions.

API — Création, suppression, déplacement

Fonction	Renvoie
<code>babét.touch(path)</code>	(true, nil) (nil, err) — crée le fichier ou met à jour <code>mtime</code>
<code>babét.mkdir(path)</code>	(true, nil) (nil, err) — toujours récursif (<code>mkdir -p</code> natif), voir ci-dessous
<code>babét.rmdir(path)</code>	(true, nil) (nil, err) — seulement dossiers vides
<code>babét.rmdirAll(path)</code>	(true, nil) (nil, err) — récursif
<code>babét.remove(path)</code>	(true, nil) (nil, err) — fichier ou symlink
<code>babét.rename(old, new)</code>	(true, nil) (nil, err)
<code>babét.chdir(path)</code>	(true, nil) (nil, err)

`chdir` mute le répertoire courant de TOUT le processus : il est interdit dès qu'un `workers.spawn` a eu lieu ((nil, err), définitif). Voir la note dans [sys](#).

Fonction	Renvoie
<code>babet.currentDir()</code>	<code>string</code> (absolu)
<code>babet.link(target, linkpath)</code>	<code>(true, nil) (nil, err)</code> — crée un lien symbolique (toujours ; pas de liens durs en <code>vl</code>)

mkdir en détail

`babet.mkdir(path)` garantit que le dossier existe à la fin de l'appel :

- **Toujours récursif** : les parents manquants sont créés en une seule fois (`mkdir -p` natif — il n'y a **pas** d'option à passer, contrairement à ce qu'une ancienne version de cette page indiquait).
- **Idempotent** : si le dossier (ou un symlink vers un dossier) existe déjà, succès silencieux.
- **Erreur claire** si un non-dossier bloque le chemin — un fichier au chemin cible ou au milieu de celui-ci : `(nil, err)` avec « Not a directory ».

```
-- crée data/, data/cache/ et data/cache/img/ d'un coup
assert(babet.mkdir("data/cache/img"))

-- le rappeler ne coûte rien : le dossier existe déjà, succès
assert(babet.mkdir("data/cache/img"))

-- un fichier bloque le chemin -> erreur propre
babet.touch("bloqueur")
local r, err = babet.mkdir("bloqueur/sous")
-- r == nil, err contient "Not a directory"
```

Signature héritée : un second paramètre booléen existe encore (`mkdir(path, ignore_if_exists)`) mais il est **déconseillé** : sur Linux, son seul effet observable est de transformer l'erreur « un symlink pendant occupe le chemin » en succès silencieux... sans rien créer. Ne l'utilise pas.

API — Manipulation de chemins

Ces fonctions opèrent sur des *strings* de chemin — elles ne touchent pas le système de fichiers.

Fonction	Renvoie	Exemple
<code>babet.getBasename(path)</code>	<code>string (nil, err)</code> — dernière composante	<code>"/etc/hostname" → "hostname"</code>
<code>babet.getPath(path)</code>	<code>string (nil, err)</code> — dossier parent (dirname)	<code>"/etc/hostname" → "/etc"</code>
<code>babet.getFilename(path)</code>	<code>string (nil, err)</code> — basename sans extension	<code>"report.tar.gz" → "report.tar"</code>
<code>babet.getExtension(path)</code>	<code>string (nil, err)</code> — extension finale	<code>"report.tar.gz" → ".gz"</code>

API — Listing et recherche

Fonction	Renvoie
<code>babet.listFiles(path)</code>	<code>table</code> (fichiers réguliers dans <code>path</code>) <code>(nil, err)</code>
<code>babet.find(path, opts)</code>	fonction iterator, voir ci-dessous
<code>babet.createFileIterator(path)</code>	fonction iterator sur les entrées du dossier

`find` accepte `opts` :

- `pattern = "*.lua"` — glob shell appliqué au nom de fichier seul.
- `recursive = true` — descend dans les sous-dossiers.
- `type = "f" / "d"` — restreint aux fichiers ou dossiers.
- `max_depth = N` — limite la profondeur de récursion.

API — Copie

Fonction	Renvoie
<code>babet.copy(src, dst, opts?)</code>	<code>(true, nil) (nil, err)</code>
<code>babet.copyTree(src, dst, opts?)</code>	<code>(true, nil) (nil, err)</code> — récursif
<code>babet.moveTree(src, dst)</code>	<code>(true, nil) (nil, err)</code>

Options de copy :

- `preserve = true` — garde mtime, mode, owner si possible.
- `overwrite = true` — remplace la destination si elle existe.

API — Hashes et checksums

Empreintes du contenu des fichiers. Résultat en **string hex minuscule** sauf mention contraire. Note le suffixe `sum` — ce sont les noms enregistrés dans `babet.*`.

Fonction	Algorithme	Notes
<code>babet.crc32sum(path)</code>	CRC-32 (IEEE 802.3)	Non cryptographique. Pour intégrité / détection d'erreur accidentelle uniquement.
<code>babet.md5sum(path)</code>	MD5	Legacy. Ne pas utiliser pour la sécurité.
<code>babet.sha1sum(path)</code>	SHA-1	Legacy. Ne pas utiliser pour la sécurité.
<code>babet.sha256sum(path)</code>	SHA-256	
<code>babet.sha384sum(path)</code>	SHA-384	
<code>babet.sha512sum(path)</code>	SHA-512	
<code>babet.sha3_256sum(path)</code>	SHA3-256	
<code>babet.sha3_384sum(path)</code>	SHA3-384	
<code>babet.sha3_512sum(path)</code>	SHA3-512	
<code>babet.blake2b512sum(path)</code>	BLAKE2b-512	

Chaque renvoie `string` (hex) ou `(nil, err)`. Le fichier est streamé, pas chargé entièrement en RAM. Les fichiers non-réguliers (dossiers, sockets, ...) sont refusés avec `(nil, "<algo>: not a regular file")`.

MD5 et SHA-1 sont exposés pour la compatibilité (Git, systèmes legacy) — ne pas les utiliser pour de nouveaux usages cryptographiques. **CRC-32 est encore plus loin du cryptographique** : un attaquant peut produire des collisions trivialement. À utiliser pour de l'intégrité / détection de changement, jamais pour de l'authentification.

API — Checksums en mémoire

Calcule un hash d'octets déjà en mémoire, sans aller-retour par un fichier temporaire. Binary-safe (les NUL intégrés sont autorisés).

Fonction	Renvoie
<code>babet.crc32(data)</code>	<code>string</code> — 8 caractères hex minuscules

Ne peut pas échouer à l'exécution (calcul pur). Les mauvais types d'argument lèvent via `luaL_error`. Comme `crc32sum`, **non cryptographique**.

Exemples rapides

```
-- Prédicats
if babet.isDir("/etc") and babet.fileExists("/etc/hostname") then
    print("taille :", babet.fileSize("/etc/hostname"))
```

```

end

-- Manipulation de chemins (string pur)
local base = babet.getBaseName("/var/log/syslog.1") -- "syslog.1"
local dir  = babet.getPath("/var/log/syslog.1")     -- "/var/log"
local ext  = babet.getExtension("/var/log/syslog.1") -- ".1"

-- Listing récursif
for entry in babet.find("/var/log",
    { pattern = "*.log", type = "f", recursive = true }) do
    print(entry)
end

-- Copie avec attributs
babet.copyTree("./src", "/tmp/backup",
    { preserve = true, overwrite = true })

-- Hash d'une tarball de release
local sha, err = babet.sha256sum("/tmp/babet-1.7.0.tar.gz")
if sha then print("SHA256 :", sha) end

-- Check d'intégrité rapide (NON cryptographique, mais rapide)
local crc = babet.crc32sum("/tmp/babet-1.7.0.tar.gz")
print("CRC32 :", crc) -- ex. "a1b2c3d4"

-- CRC32 en mémoire sur des octets arbitraires (binary-safe, NUL OK)
print(babet.crc32("abc")) -- "352441c2"
print(babet.crc32("")) -- "00000000"

-- Owner et permissions
local mode_755 = tonumber("755", 8) -- octal -> integer
assert(babet.setMode("/srv/myapp/run.sh", mode_755))
-- en root seulement :
-- assert(babet.setAttributes("/srv/myapp", 1000, 1000))

```

Contrat d'erreur

- Toutes les fonctions suivent la convention (nil, err) en cas d'échec runtime (fichier inexistant, permission refusée, etc.).
- Les mauvais **types** d'argument lèvent via `luaL_error`.
- **setAttributes(path, uid, gid)** rejette les UIDs/GIDs négatifs et les valeurs au-dessus du max `uid_t/gid_t` (typiquement $2^{32} - 1$). Sans ce check de borne haute, une valeur au-dessus de la limite serait silencieusement tronquée — pire cas vers UID 0 (root). Même logique que le module `user`.
- Sécurité des chemins : `copy`, `copyTree`, `moveTree` utilisent `lexically_relative()` et des guards `is_within()` composant par composant pour empêcher des évasions par symlink hors de l'arborescence de destination. Voir [security](#).

Décisions de design

- **Namespace plat** : `fileExists` plutôt que `fs.exists`, `getAttributes` plutôt que `fs.attr.get`. Raison purement historique — ces fonctions précèdent la convention par module. Stable maintenant ; renommer casserait tous les scripts.
- **get* / set* pour les attributs de chemin** (`getMode/setMode`, `getAttributes/setAttributes`) mais des verbes nus pour les actions FS (`remove`, `touch`, `mkdir`). Les paires `get/set` existent parce que les deux directions sont nécessaires ; les autres sont des opérations unidirectionnelles.
- **La manipulation de chemins est séparée de l'accès FS**. `getBaseName`, `getPath`, etc. n'opèrent que sur des strings — elles ne stat pas le chemin. Utilise `fileExists` si tu veux savoir si le chemin existe réellement.

- Les **hashes** ont un **suffixe sum** pour matcher les outils Unix standard (md5sum, sha256sum, etc.). Découvrable pour qui est familier de la ligne de commande.
- **copy** est **mono-fichier**, **copyTree** est **récurusif**. Lua n'a pas la surcharge de fonctions par type, donc les séparer évite l'ambiguïté "est-ce que copy('dir', 'dir') voulait dire récurusif ?".

Hors v1

- Glob matching en fonction standalone (babet.glob). Utilise find avec pattern = "*.lua" pour le cas courant.
- I/O asynchrone. Utilise plutôt workers pour le traitement parallèle de fichiers.
- BLAKE2s (seul BLAKE2b-512 est exposé via OpenSSL EVP).

babet.http — client HTTP

Un client HTTP/HTTPS simple basé sur [cpp-httpplib](#), réutilisant l'OpenSSL embarqué. Synchrone, bloquant, avec timeouts.

Pourquoi

Presque chaque script non trivial doit parler à une API HTTP. Sans client intégré, on en vient à utiliser curl via exec, avec tout l'overhead de quoting et de spawn de process. **babet.http** rend une requête accessible en une ligne, avec la vérification TLS par défaut.

API

Fonction	Renvoie
babet.http.request(opts)	response (table) (nil, err) — méthodes : GET, HEAD, OPTIONS, POST, PUT, PATCH, DELETE
babet.http.get(url, opts?)	raccourci pour request{ method="GET", url=url, ... }
babet.http.post(url, opts?)	raccourci pour request{ method="POST", url=url, ... }

Pas de raccourcis put/delete/patch en v1 (une ancienne version de cette page les annonçait à tort) : passe par request{ method = "PUT", ... }. Voir « Hors v1 ».

Table opts

Champ	Type	Défaut
url	string (requis pour request)	—
method	string — GET, HEAD, OPTIONS, POST, PUT, PATCH, DELETE	"GET"
headers	table de name = value	{}
body	string (interdit pour GET/HEAD/OPTIONS)	""
query	table { clé = valeur } — encodée URL et ajoutée à l'URL (?a=b&c=d), fusion propre avec une query déjà présente	—
timeout	number (secondes)	aucun — sans lui, les défauts internes de cpp-httpplib s'appliquent ; passe-le systématiquement
verify	boolean (vérification cert TLS)	true
ca_cert	string (chemin du fichier CA bundle)	défaut système
follow_redirects	boolean	false — le suivi est opt-in ; limite de hops interne à cpp-httpplib

Table response

```
{
  status = 200,
  body = "...",
  headers = {
    -- clés normalisées en minuscules
    ["content-type"] = "application/json",
    ["content-length"] = "42",
  },
}
```

En-têtes **dupliqués** dans la réponse : la dernière valeur gagne (une clé = une string). Conséquence connue : si un serveur envoie plusieurs **Set-Cookie**, seul le dernier est visible. Si ce besoin devient réel, headers pourra porter une table de strings pour les clés répétées — voir « Hors v1 ».

Exemples rapides

```
-- GET simple
local r, err = babet.http.get("https://api.example.com/health")
if r then
  print(r.status, r.body)
end

-- POST JSON avec header d'auth
local r, err = babet.http.post("https://api.example.com/v1/things", {
  headers = {
    ["Content-Type"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
  },
  body = babet.json.encode({ name = "widget", count = 7 }),
  timeout = 10,
})

-- Cert auto-signé (dev / CA privée)
local r = babet.http.get("https://internal.svc/", {
  ca_cert = "/etc/myapp/internal-ca.crt",
})

-- Désactiver la vérification (TESTS UNIQUEMENT)
local r = babet.http.get("https://expired.badssl.com/",
  { verify = false })
```

Contrat d'erreur

- **Mauvais types d'argument** → lève via `luaL_error`.
- **Erreurs réseau** (DNS, connect, timeout, TLS) → `(nil, "http: <description>")`.
- **body sur GET/HEAD/OPTIONS** → `(nil, "http: body not allowed for <méthode>")` — signalé, jamais ignoré en silence.
- **Méthode inconnue** → `(nil, "http: unsupported method '...')"`.
- **Les erreurs de statut HTTP ne sont pas des erreurs** — un 404 renvoie un response normal avec status = 404. La sémantique du statut est la responsabilité de l'appelant.

TLS / trust store

Babet embarque sa propre OpenSSL statique, donc il n'hérite pas automatiquement de la config `ca-certificates` de la distro. Deux mécanismes garantissent que `verify=true` fonctionne d'emblée :

1. L'OpenSSL embarquée est compilée avec `--openssldir=/etc/ssl`, couvrant Arch, Debian, Ubuntu, Alpine, Gentoo.
2. `babot.socket.connect_tls` sonde en plus les chemins de CA bundles connus (Fedora/RHEL, OpenSUSE, FreeBSD, NetBSD). **babot.http, lui, s'appuie sur les défauts OpenSSL seuls** (mécanisme 1) : sur un layout exotique où le point 1 ne suffit pas, `http` peut nécessiter `ca_cert` là où `connect_tls` fonctionne. `tools/verif_ca.lua` diagnostique les deux chemins.

Tu peux override par appel avec `ca_cert`, ou globalement via les variables d'environnement `SSL_CERT_FILE` / `SSL_CERT_DIR`. Voir [security](#) et [tls](#) pour les détails.

Décisions de design

- **Synchrone, bloquant.** Les scripts font généralement du request-response one-shot ; le sync est plus simple et suffisant. Pour beaucoup d'appels parallèles, utilise [workers](#).
- **verify=true par défaut.** Désactiver la vérification TLS doit être explicite (`verify=false`). Pas de downgrade silencieux.
- **Le statut HTTP n'est pas une erreur.** L'appelant décide si 404 ou 500 est un échec ; l'appel réseau lui-même a réussi.
- **Headers normalisés en clés minuscules** dans la réponse, pour que les lookups case-insensitive marchent directement (`r.headers["content-type"]`).
- **Redirects opt-in.** Une redirection non suivie est visible (`status = 301/302 + header location`) ; la suivre est une décision de l'appelant (`follow_redirects = true`). La limite de hops est celle de `cpp-httpplib`, pas la nôtre.

Hors v1

- Raccourcis `put` / `delete` / `patch` — couverts par `request{ method = ... }` ; sucre trivial à ajouter si le besoin se confirme.
- `opts.ca_path` (dossier de CA par appel) et `opts.max_redirects` — une ancienne version de cette page les documentait à tort.
- En-têtes de réponse répétés en table de strings (cas `Set-Cookie` multiples).
- Streaming du body de réponse (ex : pour télécharger de gros fichiers). Actuellement `body` est lu entièrement en mémoire.
- HTTP/2 / HTTP/3.
- WebSockets (utilise [socket](#) + TLS + une bibliothèque de framing Lua si nécessaire).
- Côté serveur. `cpp-httpplib` le supporte, mais exposer un serveur HTTP robuste à Lua est un design à part entière.

babot.inotify — surveillance d'événements fichier

Wrappe `inotify(7)` de Linux pour la délivrance instantanée d'événements fichier — pas de polling. Utilisé pour le rechargement à chaud de configuration, la surveillance de dossiers de dépôt, le tailing de logs, les triggers de synchronisation de fichiers, etc.

Pourquoi

Poller un dossier avec `listDir` chaque seconde est gaspilleur (CPU + lookups inode) et laggy. `inotify` est le mécanisme dédié du kernel : le kernel ne réveille ton script que quand quelque chose change réellement, avec une latence sub-milliseconde.

L'API Lua expose un contrat minimal sur un syscall notoirement délicat. Le débordement de queue est explicitement remonté (la seule chose pire que de perdre des événements silencieusement, c'est de ne pas te le dire).

API

Fonction	Renvoie
<code>babot.inotify.new()</code>	<code>watcher (userdata) (nil, err)</code>

Fonction	Renvoie
<code>w:add(path, events)</code>	<code>integer wd</code> (watch descriptor) <code>(nil, err)</code>
<code>w:remove(wd)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>w:read(timeout?)</code>	<code>table d'événements</code> <code>(nil, "timeout")</code> <code>(nil, "interrupted")</code> <code>(nil, err)</code>
<code>w:close()</code>	<code>(true, nil)</code> — idempotent

`read(timeout?)` : omis = **bloque** jusqu'au prochain événement ; `0` = **non bloquant** (rend les événements déjà en attente, sinon `(nil, "timeout")`) ; `t > 0` = attend au plus `t` secondes. Attention : cette convention diverge de celle de `workers` où `0` signifie aussi non-bloquant mais de `socket` où `set_timeout(0)` signifie infini — ici, « rends-moi ce qu'il y a maintenant » est le cas d'usage dominant.

Argument events

Une liste 1..N stricte de strings de noms d'événements. Noms acceptables :

Nom	Déclenché quand
"access"	fichier accédé
"modify"	contenu modifié
"attrib"	métadonnées changées (perms, mtime, ...)
"close_write"	un fichier ouvert en écriture est fermé
"close_nowrite"	un fd read-only est fermé
"close"	alias combiné : <code>close_write</code> ou <code>close_nowrite</code>
"open"	fichier ouvert
"moved_from"	fichier déplacé hors du dossier surveillé
"moved_to"	fichier déplacé dans le dossier surveillé
"move"	alias combiné : <code>moved_from</code> ou <code>moved_to</code>
"create"	fichier/dossier créé dans le dossier surveillé
"delete"	fichier/dossier supprimé du dossier surveillé
"delete_self"	le chemin surveillé lui-même est supprimé
"move_self"	le chemin surveillé est déplacé

Événement renvoyé par read

```
{
  wd      = 1,                -- watch descriptor
  name    = "newfile.txt",    -- "" si la cible est le chemin surveillé lui-même
  events  = { create = true, close_write = true },
  is_dir  = false,
  cookie  = 0,                -- non-zero apparie moved_from / moved_to
}
```

Événement synthétique spécial pour le **débordement de queue** :

```
{ wd = -1, events = { overflow = true } }
```

Quand tu vois ça, la queue kernel a manqué de place et des événements ont été perdus. Re-scane les chemins surveillés pour récupérer l'état.

Exemple rapide

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function() w:close(); os.exit(0) end)
```

```

while true do
    local events, err = w:read()      -- bloque jusqu'à quelque chose
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify : ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            rescan()                  -- queue saturée, récupère
        else
            handle_new_file("/srv/incoming/" .. ev.name)
        end
    end
end
end

```

Contrat d'erreur

- **read(t)** où t est NaN/Inf/négatif → lève via `luaL_error`.
- **add(path, events)** avec une table `events` non-liste (sparse, clés extra, éléments non-string) → (nil, err).
- **add sur un chemin inexistant** → (nil, "no such file or directory").
- **read :**
 - table `events` en cas de succès.
 - (nil, "timeout") si le timeout est écoulé.
 - (nil, "interrupted") si un signal géré est arrivé.
 - (nil, err) sur autres erreurs.
- **Méthodes après close** → (nil, "inotify: closed").

Décisions de design

- **Non-récurusif en v1.** `inotify(7)` lui-même n'est pas récurusif ; surveiller un arbre signifie le parcourir et appeler `add` sur chaque sous-dossier, puis gérer les événements `create` pour étendre la surveillance. C'est un vrai travail de design avec des subtilités (races entre le parcours et la délivrance d'événements, gestion du débordement) et méritera un module dédié si/quand nécessaire. Constructible en Lua pur au-dessus.
- **Liste d'événements obligatoire, pas de "all" implicite.** Surveiller tout sature la queue et force chaque événement à traverser Lua. Forcer l'appelant à choisir les événements rend le coût visible.
- **Le débordement est remonté explicitement**, jamais avalé. Le kernel a une queue finie (`/proc/sys/fs/inotify/max_queue` défaut 16384). Quand elle déborde, des événements sont perdus. La seule action appropriée est de re-scanner ; perdre silencieusement le signal flinguerait toute la fiabilité.
- **read() est interruptible par signal.** Un `read()` qui bloque pour toujours en ignorant `SIGTERM` est le bug classique de l'arrêt gracieux. Voir [signal](#).
- **IN_CLOEXEC** sur le fd : pas hérité par les sous-process exécutés, cohérent avec les sockets.

Hors v1

- Surveillance récursive. À ajouter au niveau script (walk + add) ou attendre une option `recursive = true`.
- Plusieurs watchers par process. Faisable aujourd'hui en créant plusieurs instances `inotify.new()` ; pas de registre intégré.
- Sémantique `IN_MASK_ADD` (ajouter des événements à un watch existant). Actuellement `add(path, events)` remplace le mask. Besoin rare.

babet.json — encode/décode JSON

Basé sur [nlohmann/json](#), la bibliothèque JSON C++ la plus utilisée. Gère le round-trip classique plus la sentinelle "tableau vide" explicite que les tables Lua ne peuvent pas désambigüiser de "objet vide".

Pourquoi

Le mismatch d'impédance Lua/JSON piège toujours quelqu'un : Lua ne distingue pas une liste vide d'une table vide. La plupart des bibliothèques Lua/JSON ad-hoc choisissent une option et se trompent une fois sur deux. `babet.json` est explicite : `{}` se décode en `{}` et se ré-encode en `{}` (objet), mais si tu veux `[]` tu utilises la sentinelle `babet.json.empty_array`.

API

Fonction	Renvoie
<code>babet.json.encode(value, opts?)</code>	<code>string</code> (texte JSON) <code>(nil, err)</code>
<code>babet.json.decode(text)</code>	<code>value</code> <code>(nil, err)</code>
<code>babet.json.null</code>	sentinelle — représente le <code>null</code> JSON (voir ci-dessous)
<code>babet.json.empty_array</code>	sentinelle qui s'encode en <code>[]</code>
<code>babet.json.as_array(t)</code>	<code>t</code> , marqué pour s'encoder en array JSON même ambigu/vide

Options d'encodage (table `opts`) :

- `indent = N` (entier ≥ 0) — active le pretty-print avec `N` espaces d'indentation (`0` = retours à la ligne sans espaces). Omis ou négatif : sortie **compacte**. (Une ancienne version de cette page documentait une option `pretty` qui n'a jamais existé : `indent` seul pilote tout.)

Exemple rapide

```
local J = babet.json

-- Décode
local data, err = J.decode([
{ "name": "alice", "tags": ["admin", "ops"], "active": true }
])
print(data.name, data.tags[1])

-- Encode (compact)
local out = J.encode({ a = 1, b = { 2, 3 } })
-- out == '{"a":1,"b":[2,3]}'

-- Encode pretty
local pretty = J.encode({ a = 1 }, { indent = 4 })

-- Tableau vide vs objet vide
J.encode({})          --> "{}"
J.encode(J.empty_array) --> "[]"
J.encode({ items = J.empty_array }) --> '{"items":[]}'
```

`null`, `empty_array`, `as_array`

Lua ne peut pas stocker `nil` dans une table (la clé disparaît). `babet.json.null` comble le trou :

- **Décodage** : un `null` JSON devient la sentinelle `babet.json.null` (teste avec `==`), et non `nil` — la clé reste donc présente et visible par `pairs()`.
- **Encodage** : `babet.json.null` s'encode en `null`.

`babet.json.as_array(t)` marque `t` (via une métatable interne) pour qu'encode la sérialise en array JSON, même quand la structure est ambiguë. Les sentinelles ne peuvent pas être marquées (`as_array(json.null)` lève). Les tables issues du décodage d'un array JSON vide portent déjà cette marque : le round-trip `decode("[]") → encode` redonne `[]`.

Profondeur maximale d'imbrication : **1000 niveaux** dans les deux sens (au-delà : `(nil, err)` explicite).

Contrat d'erreur

- **encode** : (nil, err) en cas d'erreur d'encodage (cycle dans le graphe, valeur qui ne mappe pas en JSON comme une fonction ou userdata, nombres NaN/Inf, etc.).
- **decode** : (nil, err) en cas d'erreur de parsing. Le message d'erreur inclut la ligne/colonne où le parse a échoué.
- **Mauvais type d'argument** → lève via `luaL_error`.

Décisions de design

- **Sentinelle `empty_array`** plutôt que de deviner depuis la structure de la table (certaines bibliothèques traitent `{1, 2, 3}` comme array et `{a=1}` comme objet, ce qui est correct, mais `{}` est ambigu). Une sentinelle explicite enlève toute surprise.
- **NaN/Inf refusés**, pas encodés silencieusement en `null`. Les consommateurs JSON gèrent `null` différemment pour les champs numériques ; mieux vaut échouer bruyamment et laisser l'appelant décider.
- **Pas d'API streaming**. Les round-trips sont bornés par la mémoire du script de toute façon ; si tu dois traiter du JSON plus gros que la RAM, utilise un vrai parser streaming dans un process séparé.

Hors v1

- Encodeur/décodeur streaming.
- Un validateur de schéma (utilise-en un en Lua pur — c'est trivial à ajouter au niveau script).

logging — logger avec niveaux (module Lua)

Un logger standard avec niveaux : `debug`, `info`, `warn`, `error`, `fatal`. Embarqué en tant que module Lua pur, chargé via `require("logging")`. Pas dans le namespace `babet` — c'est un module bibliothèque, comme `inspect`.

Pourquoi

Chaque script non trivial finit par réinventer une variante de “niveaux + timestamps + sortie fichier optionnelle”. Standardiser enlève le bikeshed et rend la sortie de log cohérente entre les scripts Babet.

API

```
local log = require("logging")
```

Fonction	Renvoie
<code>log.trace(...)</code> , <code>log.debug(...)</code> , <code>log.info(...)</code> , <code>log.warn(...)</code> , <code>log.error(...)</code> <code>log.set_level(lvl)</code>	rien — chacune émet si son niveau est au seuil courant. Ne lèvent jamais (même si le sink casse) <code>lvl</code> = nom (" <code>trace</code> "..." <code>error</code> ", insensible à la casse) ou constante (<code>log.INFO</code>). Inconnu → lève
<code>log.set_output(out)</code>	<code>out</code> = tout objet répondant à <code>:write</code> (handle <code>io.open</code> , table custom). Défaut : <code>io.stderr</code> . Mauvais type → lève
<code>log.set_color(bool)</code>	active/désactive les couleurs ANSI (défaut : off). Non-boolean strict → lève
<code>log.TRACE ... log.ERROR</code>	constantes numériques des niveaux (10, 20, 30, 40, 50)

Une ancienne version de cette page documentait un niveau `fatal` qui n'a jamais existé (appeler `log.fatal` plante : nil call) et omettait `trace` et `set_color`. Les cinq niveaux réels : `trace=10`, `debug=20`, `info=30` (seuil par défaut), `warn=40`, `error=50`.

Les messages sont préfixés `YYYY-MM-DD HH:MM:SS [NIVEAU]` (niveau paddé à 5 caractères : `[INFO ]`, `[ERROR]`). Plusieurs arguments sont joints par des espaces, comme `print`. Couleurs (opt-in via `set_color(true)`) : `trace` gris, `debug` cyan, **info sans couleur** (le cas nominal reste neutre), `warn` jaune, `error` rouge.

Exemple rapide

```
local log = require("logging")
log.set_level("info")    -- trace et debug deviennent des no-ops

log.info("démarrage, pid =", babet.pid())
log.warn("tentative", 3, "sur", 5)
log.error("connexion échouée :", err)

-- Optionnel : router vers un fichier (objet :write-able)
local f = io.open("/var/log/myapp.log", "a")
if not f then
    log.error("impossible d'ouvrir le log ; on reste sur stderr")
else
    log.set_output(f)
end
```

Sortie :

```
2026-06-11 14:32:01 [INFO ] démarrage, pid = 12345
2026-06-11 14:32:05 [WARN ] tentative 3 sur 5
2026-06-11 14:32:05 [ERROR] connexion échouée : timeout
```

Contrat d'erreur

- **set_level / set_output / set_color** : argument invalide (niveau inconnu, objet sans :write, non-boolean) → lève via error() — c'est un bug du programmeur, pas une condition d'exécution.
- **Les fonctions d'émission ne lèvent jamais** : l'écriture est entourée d'un pcall — un sink qui casse en cours de route (disque plein, fichier fermé) fait perdre le message en silence, jamais crasher le script.

Décisions de design

- **Module Lua pur**. Pas besoin de C++, et être en Lua signifie que les scripts peuvent le monkey-patcher (ex : ajouter une sortie format JSON) au niveau appelant sans recompiler Babet.
- **Cinq niveaux, trace à error — pas de fatal**. error + os.exit(1) explicite vaut mieux qu'un niveau qui tue le process en douce ; et cinq niveaux suffisent à presque tout le monde.
- **Couleurs opt-in, info toujours neutre**. Le cas nominal ne doit pas crier ; seuls les écarts (warn, error) se voient.
- **Sink global unique**. Plusieurs loggers / loggers hiérarchiques / niveaux par module est un piège de feature creep. Si tu as besoin de plusieurs destinations, écris un wrapper.

Hors v1

- Sortie JSON / logging structuré. Facile à bricoler au niveau script si nécessaire.
- Rotation des logs. Utilise logrotate(8) sur le fichier de sortie.
- Sortie asynchrone / buffered. Optimisation prématurée pour l'usage typique.

babet.signal — signaux POSIX

Enregistre des callbacks Lua pour les signaux POSIX (SIGTERM, SIGINT, SIGHUP, ...) pour que les scripts long-running puissent s'arrêter proprement ou recharger leur configuration à la demande.

Pourquoi

Les scripts Babet long-running (bots, daemons, watchers) doivent gérer les signaux correctement :

- SIGTERM de systemctl stop devrait déclencher un arrêt propre.
- SIGINT de Ctrl-C devrait faire la même chose.

- `SIGHUP` est le signal conventionnel “reload config”.

Sans handlers explicites, ces signaux tuent juste le process, laissant des fichiers ouverts, des bases de données mi-écrites, et des enfants orphelins.

API

Fonction	Renvoie
<code>babet.signal.handle(name, fn)</code>	<code>(true, nil) (nil, err)</code> — installe le callback Lua
<code>babet.signal.ignore(name)</code>	<code>(true, nil) (nil, err)</code> — met à <code>SIG_IGN</code>
<code>babet.signal.default(name)</code>	<code>(true, nil) (nil, err)</code> — retour au défaut OS

Une ancienne version de cette page documentait aussi `kill`, `list` et `is_pending` : ils n’existent pas — voir « Hors v1 ».

Noms de signaux acceptés (string, casse sensible) :

`"TERM"`, `"INT"`, `"HUP"`, `"USR1"`, `"USR2"`, `"PIPE"` — les six signaux de la v1 (une ancienne version de cette page en listait quatorze ; étendre la table est trivial, voir « Hors v1 »). `KILL` et `STOP` ne pourront jamais y figurer — POSIX interdit de les attraper.

Exemple rapide

```
local sig = babet.signal
local running = true

sig.handle("TERM", function()
    print("SIGTERM reçu, arrêt en cours")
    running = false
end)
sig.handle("INT", function()
    print("SIGINT reçu, arrêt en cours")
    running = false
end)
sig.handle("HUP", function()
    print("SIGHUP reçu, rechargement config")
    cfg = load_config()
end)

while running do
    local line, err = sock:recv_line(1.0)
    if not line then
        if err == "interrupted" then
            -- Un signal géré est arrivé pendant le recv ; le
            -- callback s'est déjà exécuté. Re-vérifier la condition.
        else
            break
        end
    else
        process(line)
    end
end

print("sortie propre")
```

Comment les callbacks s’exécutent

Quand un signal arrive :

1. Le kernel positionne un flag pending (aucun code Lua ne tourne depuis le signal handler — restrictions `async-signal-safe`).
2. Si le script est dans un appel bloquant (`recv`, `sleep`, `inotify.read`, `accept`, ...), l'appel renvoie (`nil`, `"interrupted"`).
3. Avant de renvoyer, Babet dispatche tous les callbacks en attente dans l'ordre d'enregistrement, dans le thread Lua principal.
4. Le script continue normalement — le callback a pu modifier des globales (`running = false` est le pattern typique).

Ça veut dire que les callbacks s'exécutent toujours en contexte Lua, jamais depuis l'intérieur d'un signal handler. Ils peuvent utiliser n'importe quelle fonctionnalité Lua (pas de restrictions `async-signal-safe`).

Interaction avec `debug.sethook` (audit v21) : hors des appels bloquants, le dispatch s'appuie sur un debug hook Lua (`lua_sethook`, déclenché périodiquement par compteur d'instructions). Conséquences : un `debug.sethook(...)` posé par le script **écrase** ce mécanisme — les callbacks ne seront plus dispatchés que lorsque le script entre dans un appel bloquant **babet** — et réciproquement, le premier `signal.handle(...)` remplace un hook utilisateur déjà installé. Évite de mélanger `babet.signal` et `debug.sethook` dans le même script.

Contrat d'erreur

- **Nom de signal inconnu** → (`nil`, `"signal: unknown name 'XYZ'"`).
- **Appels depuis un worker thread** → lève via `luaL_error` (`"signal.handle: signal handlers can only be configured from the main thread, not from a worker"`). C'est un bug de structure du script, pas une condition d'exécution — d'où la levée plutôt qu'un (`nil`, `err`). Les signaux sont globaux au process ; seul le thread principal les gère.
- **Mauvais types d'argument** → lève via `luaL_error`.
- **Le callback ne reçoit aucun argument** (installe un callback par signal si tu dois les distinguer) et **ses erreurs sont avalées en silence** : entoure ton code d'un `pcall` si tu veux les observer.

Décisions de design

- **Callbacks tournent dans le thread Lua principal, pas depuis le signal handler**. Les règles `async-signal-safety` de POSIX interdisent la plupart des opérations utiles depuis un vrai handler. En marquant le signal comme pending et en dispatchant depuis le prochain point sûr, les callbacks peuvent faire tout ce que Lua sait faire.
- **Les appels bloquants renvoient "interrupted" sur un signal géré**. Sans ça, un `recv_line` attendant sur le réseau bloquerait jusqu'au timeout malgré l'arrivée de `SIGTERM`. Avec ça, l'arrêt peut se faire en millisecondes.
- **Pas de ré-entrance**. Pendant qu'un callback tourne, les signaux additionnels sont mis en file et dispatchés ensuite.
- **Interdit dans les worker threads**. Les signaux sont process-wide ; seul un thread peut sensément les gérer. Les workers doivent utiliser des channels pour la communication inter-thread.

Hors v1

- `kill(pid, name)`, `list()` et `is_pending()` — une ancienne version de cette page les présentait à tort comme disponibles. Contournements : envoyer un signal = `babet.exec("kill", { "-TERM", tostring(pid) })` ; la liste des noms supportés est ci-dessus.
- Étendre la table des signaux (`QUIT`, `ALRM`, `CHLD`, `WINCH`, ...) — trivial si un besoin réel se présente.
- `sigprocmask` / masquage fin de signaux. Pas souvent nécessaire au niveau script.
- Intégration avec `signalfd` pour le polling. Le modèle de dispatch actuel est plus simple et couvre les cas typiques.

babet.socket — sockets TCP client et serveur

Sockets TCP bas niveau avec timeouts, conscience des signaux, et un petit set de helpers haut niveau (`recv_line`, `recv_all`). Base pour tout protocole personnalisé — le bot IRC, un daemon JSON-over-TCP, etc. Voir [tls](#) pour la variante chiffrée.

Pourquoi

Pour les besoins HTTP courants il y a [http](#). Mais beaucoup de scripts doivent parler un protocole line-oriented personnalisé (IRC, Redis, daemons de style telnet), où un vrai socket TCP avec timeouts est l'outil approprié. Lua n'a pas de TCP intégré, et `LuaSocket` est une dépendance séparée ; `babel.socket` rend les patterns courants accessibles en une ligne.

API

Constructeurs

Fonction	Renvoie
<code>babel.socket.connect(host, port, timeout?)</code>	<code>socket</code> <code>(nil, err)</code>
<code>babel.socket.listen(host, port, backlog?)</code>	<code>server_socket</code> <code>(nil, err)</code>

- `timeout` : secondes (float), optionnel — défaut infini (bloquant). C'est le budget des tentatives TCP **après résolution DNS** : deadline **globale**, toutes adresses A/AAAA confondues. Si le DNS renvoie plusieurs adresses (dual-stack typique), les tentatives successives se partagent le même budget — jamais `timeout` par adresse. Au dépassement : `(nil, "timeout")`. Limite connue : la résolution DNS elle-même (`getaddrinfo`, synchrone) n'est pas couverte par cette deadline ni interruptible par signal — un resolver lent peut ajouter son propre délai en amont.
- `backlog` : entier > 0, optionnel — défaut 16. Taille de la file d'attente `listen(2)`.

TLS : `babel.socket.connect_tls(host, port, opts?)` et `s:starttls(opts?)` existent aussi — ils ont leur page dédiée, voir [tls](#).

Un `connect` — avec ou sans `timeout` — interrompu par un signal géré via [signal](#) renvoie `(nil, "interrupted")` après avoir dispatché le callback, comme `recv/accept`.

Méthodes (socket client)

Méthode	Renvoie
<code>s:send(data)</code>	<code>integer</code> octets envoyés <code>(nil, err)</code> — envoie tout (boucle interne sur les écritures partielles)
<code>s:recv(n, timeout?)</code>	<code>string</code> <code>(nil, "timeout")</code> <code>(nil, "interrupted")</code> <code>(nil, err)</code>
<code>s:recv_line(timeout?)</code>	<code>string</code> (sans <code>\n</code>) <code>(nil, ...)</code>
<code>s:recv_all(timeout?)</code>	<code>string</code> (lit jusqu'à EOF) <code>(nil, ...)</code>
<code>s:set_timeout(seconds)</code>	positionne le <code>timeout</code> par défaut (0 = infini). NaN/Inf/négatif → <code>(nil, err)</code> ; plancher interne 1 ms pour les valeurs positives minuscules
<code>s:close()</code>	idempotent
<code>s:peer()</code>	<code>table {host, port}</code> <code>(nil, err)</code> — endpoint distant
<code>s:sockname()</code>	<code>table {host, port}</code> <code>(nil, err)</code> — endpoint local

`recv_line` lit jusqu'à trouver `\n` (LF). CR est strippé s'il est présent (`\r\n` → renvoyé sans le `\r` final). A un cap dur de 8 MiB pour empêcher un DoS via une ligne unique sans fin.

Méthodes (socket serveur)

Méthode	Renvoie
<code>srv:accept(timeout?)</code>	<code>socket</code> <code>(nil, ...)</code>
<code>srv:close()</code>	idempotent

Exemples rapides

Client TCP echo

```
-- timeout de 5 s pour le connect COMPLET (toutes adresses DNS
-- confondues) ; omets-le pour un connect bloquant.
local s = assert(babet.socket.connect("localhost", 4000, 5))

s:send("hello\n")
local reply, err = s:recv_line()
print("reçu :", reply)
s:close()
```

Serveur line-protocol simple avec arrêt propre

```
local sig = babet.signal
local srv = assert(babet.socket.listen("0.0.0.0", 4000))
local running = true
sig.handle("TERM", function() running = false end)
sig.handle("INT", function() running = false end)

while running do
    local client, err = srv:accept(1.0)      -- timeout 1 s
    if not client then
        if err == "timeout" or err == "interrupted" then
            -- boucle et re-vérifie `running`
        else
            io.stderr:write("accept : ", err, "\n"); break
        end
    else
        repeat
            local line = client:recv_line(30)
            if line then client:send("tu as dit : " .. line .. "\n") end
        until not line
        client:close()
    end
end
srv:close()
```

Contrat d'erreur

- **Erreurs de connect** (DNS, refusé, réseau) → (nil, "socket: ..."). Dépassement du timeout → (nil, "timeout") (nu). Signal géré pendant le connect → (nil, "interrupted").
- **Erreurs de bind** (port utilisé, permission) → (nil, "socket: ...").
- **recv** :
 - string (n'importe quelle longueur jusqu'à n demandé).
 - String vide "" quand le peer ferme proprement. Pas une erreur.
 - (nil, "timeout") si pas de données dans le timeout.
 - (nil, "interrupted") si un signal géré est arrivé.
 - (nil, "line too long") pour `recv_line` au-delà du cap 8 MiB.
 - (nil, err) sur autres erreurs.
- **send** : peut renvoyer moins d'octets que demandé. Encapsule dans une boucle si tu dois envoyer une quantité fixe (ou utilise des sémantiques `recv_all`-symétriques dans ton protocole).
- **Méthodes après close** → (nil, "socket: closed").
- **Timeouts NaN/Inf**, timeouts négatifs → lève via `luaL_error`.

Décisions de design

- **Single-threaded, I/O bloquant.** Pas de `select/poll` exposé — utilise `workers` pour les connexions concurrentes, ou des timeouts courts dans une boucle de polling pour les cas simples.
- **recv_line a un cap 8 MiB.** Protège contre un peer qui envoie des mégaoctets sans jamais envoyer `\n`. Configurable plus tard si un cas d'usage demande plus.
- **Blocage conscient des signaux.** Tous les `recv*`, `accept` — et `connect`, y compris sans timeout — renvoient `"interrupted"` sur signal géré, pour que le script puisse sortir proprement.
- **Timeout de connect global, pas par adresse** (audit v21). La deadline est créée une fois pour l'appel entier ; un host dual-stack dont la première adresse traîne ne multiplie pas le délai. Contrepartie assumée : une adresse de secours n'a que le budget restant.
- **recv renvoie "" sur fermeture propre, pas nil.** Permet aux scripts de distinguer “le distant est parti” (“”) de “pas encore de données” (“`timeout`”) sans inventer encore une autre sentinelle.

Hors v1

- Sockets UDP. Facile à ajouter additivement (`babet.socket.udp.*`).
- Sockets domaine Unix. Même raisonnement.
- Multiplexage natif (`select/poll/epoll` exposé à Lua). Utilise plutôt les workers ou le polling à timeout court.

babet.sqlite — base de données SQL embarquée

Wrappe SQLite 3.53.1 (embarquée), le moteur de base de données le plus déployé au monde. Persistance zero-config pour tout script qui a besoin de plus qu'un fichier JSON mais moins qu'un serveur de base de données.

Pourquoi

Un script qui doit garder un état entre les runs (la seen-list d'un bot, la progression d'un scraper, des métriques accumulées) ne devrait pas avoir à tirer PostgreSQL ou inventer son propre format de fichier. SQLite est exactement ce qu'il faut à cette échelle : fichier unique, transactionnel, rapide, pas de daemon.

L'API Lua reflète l'API C de SQLite de près (`open` / `exec` / `prepare` / `step` / `finalize`) avec des défauts plus sûrs et un retour d'erreurs Lua-friendly.

API

Ouverture et fermeture

Fonction	Renvoie
<code>babet.sqlite.open(path, opts?)</code>	<code>db (userdata) (nil, err)</code>
<code>db:close()</code>	<code>(true, nil)</code> — idempotent

`opts` :

Champ	Type	Défaut
<code>wal</code>	boolean — active le journal WAL	false
<code>busy_timeout</code>	integer ms — retry automatique pendant cette durée sur une db verrouillée	0 = aucun (SQLITE_BUSY immédiat)

Une ancienne version de cette page documentait aussi `opts.readonly` et `opts.foreign_keys` : ils n'existent pas — voir « Hors v1 ».

Chemin spécial `":memory:"` ouvre une base en mémoire (perdue à la fermeture). À utiliser pour les tests ou le traitement transitoire.

Exécution et requêtes

Fonction	Renvoie
<code>db:exec(sql)</code>	<code>(true, nil) (nil, err)</code> — multi-statements acceptés
<code>db:exec(sql, params)</code>	idem, avec paramètres bindés
<code>db:query(sql, params?)</code>	<code>stmt</code> (itérateur) <code>(nil, err)</code> — un seul statement

`db:query` renvoie un **itérateur callable**, pas une table : chaque appel rend la row suivante (table dont les clés sont les noms de colonnes), puis `nil` une fois épuisé — l'usage idiomatique est `for row in db:query(...) do`. Les ressources sont libérées à l'épuisement ou au ramasse-miettes ; `stmt:close()` permet de libérer plus tôt (itération abandonnée en cours de route).

Un SQL multi-instructions passé à `query` rend `(nil, "sqlite: query supports only one statement; ...")` — pour du multi-statement, c'est `exec`.

Mapping de types

Type SQLite	Type Lua
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string (binary-safe)
NULL	lecture : clé absente de la row (<code>row.col == nil</code> , mais <code>pairs()</code> ne la voit pas) ; écriture : utilise le littéral SQL <code>NULL</code> (pas de sentinelle en v1)

Exemples rapides

```
local db = assert(babet.sqlite.open("state.db", { wal = true }))

-- Schéma (multi-statements : exec)
db:exec([[
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        active INTEGER DEFAULT 1
    );
]])

-- Insert avec paramètres
assert(db:exec("INSERT INTO users (name) VALUES (?)", { "alice" }))

-- Dernier rowid : via SQL (pas de méthode dédiée en v1)
for row in db:query("SELECT last_insert_rowid() AS id") do
    print("inséré avec id", row.id)
end

-- Requête : query rend un ITÉRATEUR
for row in db:query("SELECT id, name FROM users WHERE active = ?",
    { 1 }) do
    print(row.id, row.name)
end

-- Transaction : via SQL (pas de wrapper en v1)
```

```

db:exec("BEGIN")
local ok1 = db:exec("UPDATE users SET active = 0 WHERE name = ?",
                    { "alice" })
local ok2 = db:exec("UPDATE users SET active = 1 WHERE name = ?",
                    { "bob" })
if ok1 and ok2 then
    db:exec("COMMIT")
else
    db:exec("ROLLBACK")
end

db:close()

```

Contrat d'erreur

- Toutes les erreurs runtime → (nil, "sqlite: <description>") avec le code d'erreur SQLite dans le message.
- **exec(sql)** avec des **placeholders** mais sans argument **params** → (nil, "sqlite: SQL contains placeholders but no params table provided; ...") — erreur explicite plutôt que binding NULL silencieux. Le garde s'applique à **chaque** statement d'un SQL multi-instructions (audit v21).
- **Placeholders supportés** : ? (positionnels, bindés par params[1], params[2], ...) et :name / @name / \$name (nommés, bindés par params.name — le préfixe est ignoré). Les placeholders **numérotés ?NNN ne sont pas supportés** : le binder les traiterai comme un paramètre « nommé » dont le nom est "NNN" (clé string), un piège plus qu'une fonctionnalité. Utilise ? ou :name.
- **Clés sparses dans params** (ex : {[1] = "a", [3] = "c"}) → (nil, err).
- **String BLOB vide** → stockée correctement en tant que BLOB vide (anciennement buggé en pré-1.5).
- **Méthodes après close** → (nil, "sqlite: connection closed").
- **Mauvais types d'argument** → lève via lua_error.

Décisions de design

- **WAL opt-in, pas par défaut**. WAL est strictement meilleur pour la plupart des cas d'usage, mais crée des fichiers sidecar *.wal et *.shm que certains utilisateurs trouvent surprenants. L'opt-in garde le défaut sans surprise, mais tout daemon long-running devrait passer wal=true.
- **Placeholders sans params est une erreur**, pas un bind NULL silencieux. Attrape une classe de bugs proches de l'injection tôt.
- **Erreurs renvoyées, pas levées**. Même un SQL malformé renvoie (nil, err). Les scripts qui ne vérifient pas sont bruyants mais pas catastrophiques.

Hors v1

Les éléments suivants figuraient à tort comme disponibles dans une ancienne version de cette page — ils relèvent du design initial et ne sont **pas implémentés** : db:prepare / stmt:exec / stmt:finalize (statements préparés réutilisables — query re-prépare à chaque appel), db:transaction(fn) / db:in_transaction() (contournement : exec("BEGIN"/"COMMIT"/"ROLLBACK"), cf. exemple), db:last_insert_rowid() / db:changes() (contournement : SELECT last_insert_rowid() / SELECT changes()), opts.readonly / opts.foreign_keys (contournement : PRAGMA foreign_keys = ON via exec), et la sentinelle db.NULL en écriture.

- I/O streaming BLOB (sqlite3_blob_open). Utilise la sérialisation en string si tu rentres en mémoire.
- Virtual tables / FTS5 / R-Tree. Disponibles via SQL brut si le SQLite embarqué est compilé avec ; pas encore d'API niveau Lua.
- API de backup (sqlite3_backup_init). Pour l'instant, VACUUM INTO 'backup.db' marche en one-liner.

Pour une couche de plus haut niveau type ORM, construis-la en Lua au-dessus de cette API.

babet strings — manipulation de chaînes

Une seule fonction : découper une chaîne par séparateur en table. Précède la convention par sous-namespace, vit directement sur `babet`.

Pourquoi

Découper une chaîne par délimiteur est l'une des opérations string les plus courantes, et la `stdlib` Lua n'en a pas par défaut (`string.gmatch` marche mais demande d'échapper les patterns). Une fonction dédiée est plus découvrable et évite le piège du pattern escaping.

API

Fonction	Renvoie
<code>babet.split(s [, sep [, max_splits]])</code>	<code>table</code> (array) des sous-chaînes

- `s` : la chaîne à découper. Binaire-safe : les octets NUL sont préservés.
- `sep` : séparateur optionnel — **un seul caractère**, littéral (pas de pattern). Une chaîne vide, ou l'omission de l'argument, bascule en **mode caractères** : `s` est découpée en caractères individuels. Plus d'un caractère → lève.
- `max_splits` : nombre maximal de coupes, optionnel (défaut -1 = illimité). Le reste non découpé atterrit dans le dernier élément ; 0 renvoie donc { `s` }. Ignoré en mode caractères.

Si `sep` n'apparaît pas dans `s`, le résultat est une table à un élément contenant `s` en entier.

Cas limites de la chaîne vide (comportement historique, figé et testé) :

- `babet.split("", sep)` renvoie { "" } — une entrée vide, pas une table vide.
- `babet.split("")` (mode caractères) renvoie {} — table vide.

Exemple rapide

```
local parts = babet.split("a,b,c,d", ",")
-- parts == { "a", "b", "c", "d" }

local one = babet.split("hello", ",")
-- one == { "hello" }

-- Mode caractères (sep omis ou vide)
local chars = babet.split("abc")
-- chars == { "a", "b", "c" }

-- Nombre de coupes borné : le reste dans le dernier élément
local kv = babet.split("clé,val,ue", ",", 1)
-- kv == { "clé", "val,ue" }
```

Contrat d'erreur

- Mauvais type d'argument → lève via `luaL_error`.
- `sep` de plus d'un caractère → lève.
- `max_splits` non entier ou `< -1` → lève.
- Sinon, réussit toujours.

Décisions de design

- **Séparateur littéral, pas un pattern Lua**. Le cas courant est le découpage de données CSV-like, où on veut que `.` signifie un vrai point, pas “n'importe quel caractère”. Si on a besoin de pattern matching, on retombe sur `string.gmatch`.

- **Les entrées vides sont préservées.** "a,,b" se découpe en {"a", "", "b"}. Les consommateurs qui veulent filtrer les strings vides le font en une ligne de Lua.
- **split("", sep) renvoie { "" }.** C'est la conséquence naturelle de la règle "entrées vides préservées" (zéro séparateur trouvé → un élément, la chaîne entière — qui se trouve être vide). Comportement observable depuis la v1, donc figé plutôt que changé.
- **Mode caractères quand sep est omis ou vide.** Il n'y a pas de séparateur par défaut : sans séparateur, la seule interprétation cohérente d'un découpage est caractère par caractère. Précision : le découpage se fait par **octets**, pas par points de code Unicode — un caractère UTF-8 multi-octets sera éclaté.

Hors v1

- Découpage basé pattern (avec règles d'échappement). Utilise `string.gmatch` si nécessaire.
- Séparateurs multi-caractères.
- Miroir `joinTable(t, sep)` — `table.concat` existe déjà dans la stdlib.

babet sys — utilitaires système

Helpers d'inspection processus et hôte : PID, hostname, infos kernel, lookup d'exécutable, variables d'environnement.

Pourquoi

La stdlib Lua a `os.getenv` et `os.execute`, mais rien pour le PID, le hostname, les infos kernel, ou un `which` portable. Ce sont des besoins de scripting universels qui ne méritent pas un détour par `popen`.

Ces fonctions vivent directement sur **babet** (plat, pas de sous-table) parce qu'elles précèdent la convention par module adoptée pour les ajouts récents.

API

Fonction	Renvoie
<code>babet.pid()</code>	<code>integer</code> — PID du processus courant
<code>babet.hostname()</code>	<code>string</code> <code>(nil, err)</code> — hostname système
<code>babet.uname()</code>	table avec <code>sysname</code> , <code>nodename</code> , <code>release</code> , <code>version</code> , <code>machine</code>
<code>babet.which(cmd)</code>	<code>string</code> (chemin complet dans PATH) <code>(nil, "which: '<cmd>' not found in PATH")</code>
<code>babet.env(name)</code>	<code>string</code> <code>nil</code> — comme <code>os.getenv</code> , en cohérent
<code>babet.setenv(name, value)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>babet.getMemoryUsage()</code>	<code>integer</code> — mémoire de la VM Lua en octets (après un GC complet)
<code>babet.getDetailedMemoryUsage()</code>	<code>(integer, integer)</code> — actuellement les deux valeurs sont égales au compteur GC en octets ; gardé en deux retours pour la stabilité d'API

Constantes runtime

La même table **babet** expose aussi des constantes qui décrivent le binaire Babet qui exécute le script. Utile pour le logging, pour conditionner une feature à une version minimum, ou pour sanity-checker le runtime.

Constante	Type	Exemple
<code>babet.VERSION</code>	<code>string</code>	"2.1.1"
<code>babet.VERSION_MAJOR</code>	<code>integer</code>	2
<code>babet.VERSION_MINOR</code>	<code>integer</code>	1
<code>babet.VERSION_PATCH</code>	<code>integer</code>	1

La version string est ce que `babet --version` affiche. Les composantes integer sont là pour faire des comparaisons programmatiques sans parser la string.

```
-- Logger le runtime
print("tourne sous Babet " .. babet.VERSION)

-- Conditionner une feature à une version minimum
local need_minor = 1
if babet.VERSION_MAJOR < 2
  or (babet.VERSION_MAJOR == 2 and babet.VERSION_MINOR < need_minor) then
  error("ce script nécessite Babet >= 2." .. need_minor)
end
```

Exemple rapide

```
print("Sur :", babet.hostname(), "PID :", babet.pid())

local u = babet.uname()
print("Kernel :", u.sysname, u.release, u.machine)

-- Trouver un binaire
local sh, err = babet.which("bash")
if sh then
  babet.exec({ sh, "-c", "echo hello" })
end

-- Lire/écrire env
print("HOME :", babet.env("HOME"))
babet.setenv("MY_VAR", "value")

-- Introspection mémoire (force un GC complet d'abord)
print("Mémoire VM Lua :", babet.getMemoryUsage(), "octets")
local a, b = babet.getDetailedMemoryUsage()
print("Détaillé :", a, b)
```

Contrat d'erreur

- `pid()` : ne peut pas échouer, retourne toujours un integer.
- `hostname()` / `which()` : (value, nil) en succès, (nil, err_string) en échec.
- `env(name)` : value si défini, nil sinon. Lève seulement sur un argument non convertible en string (nil, table...) — un nombre est converti (`env(42)` cherche "42").
- `setenv(name, value)` : (true, nil) en succès, (nil, err) en échec (rare — généralement OOM ou nom invalide).
- `uname()` : ne peut pas échouer sur un système supporté, renvoie toujours la table complète.
- **Mauvais type d'argument** (ex : `which({})`) → lève via `luaL_error` après coercion string par convention Lua. Passe une table ou booléen pour forcer une vraie erreur.

Décisions de design

- `env(name)` renvoie nil, pas "", pour les variables non définies. Cohérent avec `os.getenv`. Les tests doivent utiliser `env("X") == nil`, pas `env("X") == ""`.
- `which` renvoie le chemin absolu, pas juste "oui/non". C'est plus utile : on peut `exec` le chemin renvoyé directement. Pour un check booléen, utilise `which(cmd) ~= nil`.
- `uname()` renvoie une table, pas plusieurs valeurs. Reste forward-compatible si un champ est ajouté un jour (ex : domainname).
- `setenv` et les workers ne se mélangent pas (audit v21 ; règle appliquée par le runtime depuis la revue croisée). POSIX ne synchronise pas `setenv/getenv` entre threads : un `babet.setenv` pendant qu'un worker

— ou le runtime lui-même (résolution DNS, locale, `exec`) — lit l’environnement est une course de données. `babet.setenv` (et `babet.chdir`, dont le répertoire courant est lui aussi partagé par tout le processus) rendent donc (`nil`, `err`) dès qu’un `workers.spawn` a eu lieu — définitivement, même après `join`, même si le `spawn` a échoué. La transition est sérialisée par le même verrou que les mutations : aucun worker ne peut naître pendant un `setenv/chdir`. Fixe environnement et répertoire courant **avant** le premier worker.

Hors v1

Additif — pourrait être ajouté plus tard :

- `unsetenv` — retirer une variable héritée n’est pas possible en v1 (`setenv(name, nil)` **lève** : la valeur doit être une string ; une valeur vide reste une variable définie). Et depuis le verrou `workers`, toute mutation d’env est de toute façon interdite après le premier `spawn`.
- `getuid` / `getgid` / `getppid` / `getlogin`.
- Dump complet de l’environnement en table.

Pour les lookups d’utilisateurs (UID → nom, etc.) voir le module dédié `user` — il utilise NSS, qui est le bon backend pour cette question.

babet tables — helpers de manipulation de tables

Deux helpers qui comblent des manques évidents de la bibliothèque standard Lua : une vraie deep copy, et un merge multi-sources.

Pourquoi

La stdlib Lua n’a pas de deep copy intégrée ni de merge multi-tables. Les deux sont des besoins courants : merger une config defaults + overrides, snapshotter une table avant mutation, copier une arborescence d’options. Les écrire à chaque script est répétitif et source d’erreurs (oublier la metatable, les cycles, etc.).

Ces deux fonctions vivent directement sur la table `babet` (pas de sous-namespace) parce qu’elles précèdent la convention par module adoptée pour les ajouts récents.

API

Fonction	Renvoie
<code>babet.mergeTables(t1, t2, ...)</code>	nouvelle table — clés string mergées (le dernier écrit gagne), clés numériques append en ordre
<code>babet.deepCopyTable(t)</code>	nouvelle table , copiée récursivement

`mergeTables` est variadique — **deux tables minimum** (moins : lève), autant que tu veux au-delà. Les entrées ne sont pas mutées. Le merge a deux règles :

- **Clés string** : les tables suivantes écrasent les précédentes à la même clé (le dernier écrit gagne). Le remplacement est **shallow** : une valeur-table écrase la précédente **en bloc**, il n’y a pas de merge récursif des sections imbriquées. Et les valeurs sont **référéncées, pas copiées** — le résultat partage ses sous-tables avec les entrées ; combine avec `deepCopyTable` si tu veux un résultat indépendant.
- **Clés numériques (partie array)** : concaténées dans l’ordre où les tables sont passées, puis dans l’ordre 1..n à l’intérieur de chaque table. Donc `mergeTables({1, 2}, {3, 4})` donne `{1, 2, 3, 4}`, pas `{3, 4}`.

`deepCopyTable` copie les tables imbriquées récursivement. Les cycles sont détectés (pas de boucle infinie), et les sous-tables partagées ne sont copiées qu’une fois (la structure est préservée, pas dupliquée). La **metatable** de chaque table copiée est posée telle quelle sur la copie (référence partagée — la metatable elle-même n’est pas dupliquée). Les valeurs non-table (nombres, strings, booléans, fonctions, userdata) sont référéncées telles quelles. Profondeur maximale : **75 niveaux** (au-delà : lève "Table is too deep to copy (max depth 75 exceeded)").

Exemple rapide

```
-- Clés string : le dernier écrit gagne
local defaults = { port = 8080, host = "localhost", debug = false }
local user_cfg = { port = 9000, debug = true }
local cfg = babet.mergeTables(defaults, user_cfg)
-- cfg.port == 9000      (user_cfg a override)
-- cfg.host == "localhost"
-- cfg.debug == true

-- Clés numériques : append en ordre
local a = { 1, 2 }
local b = { 3, 4 }
local merged = babet.mergeTables(a, b)
-- merged == { 1, 2, 3, 4 }  (PAS { 3, 4 })

-- Deep copy : snapshot avant mutation
local snapshot = babet.deepCopyTable(cfg)
cfg.port = 9999          -- N'AFFECTE PAS snapshot
print(snapshot.port)     -- toujours 9000
```

Contrat d'erreur

- **Mauvais type d'argument** (passer une non-table à l'une ou l'autre des fonctions) → lève via `luaL_error`.
- Aucune des deux fonctions ne renvoie d'erreur via `(nil, err)`. Elles réussissent ou lèvent.

Décisions de design

- **Les valeurs shallow sont référencées, pas deep-copiées**, dans `deepCopyTable`. Copier des fonctions ou des userdata n'a pas de sens, et copier des valeurs immuables (nombres, strings, booleans) ne change rien. Seules les tables sont dupliquées récursivement.
- **Les metatables ne sont pas copiées** dans `deepCopyTable`. C'est un choix conscient : préserver une metatable à travers une copie peut surprendre l'appelant (la copie déclenche encore des callbacks `__index` qui mutent l'original). Si tu dois copier avec metatable, fais-le explicitement avec `setmetatable(babet.deepCopyTable(t), getmetatable(t))`.
- **mergeTables est shallow**. Si deux tables sources partagent une sous-table à la même clé string, le résultat référence la dernière telle quelle — il ne récurse pas dedans. Combine avec `deepCopyTable` si tu veux une sémantique merge-puis-isole.
- **Les clés numériques sont append, pas écrasées**. Ce choix vient du cas d'usage Lua le plus courant : combiner des tables type liste (`{1, 2} + {3, 4} → {1, 2, 3, 4}`). Si tu voulais une sémantique d'écrasement sur les clés numériques, construis une table de destination et assigne explicitement.

Hors v1

Additif — pourrait être ajouté plus tard sans casser le SemVer :

- Un `deepMergeTables` qui récurse dans les tables imbriquées quand les deux sources ont une table à la même clé. Demande courante, mais la sémantique autour des listes vs maps devient ambiguë, d'où pas dans v1.
- Un `equalsTables` pour comparaison structurelle profonde. Facile à écrire en Lua pur, n'a pas émergé comme besoin réel.

babet time — horloges monotone et temps réel

Deux horloges avec précision sub-seconde et un `sleep` qui respecte les unités `s/ms/us` ainsi que les signaux. Comble les trous de `os.time` / `os.clock`. Les trois fonctions historiques vivent directement sur `babet` ; les utilitaires de dates et durées vivent sous la sous-table `babet.time` (qui expose aussi les trois premières en alias).

Pourquoi

`os.time()` ne donne que des secondes, et `os.clock()` mesure le temps CPU, pas le temps réel. Aucun des deux n'est approprié pour mesurer des durées réelles (latence de requête, backoff de retry, timeouts). Et `os` n'a pas de `sleep` portable sub-seconde.

`babet.monotonic()` et `babet.now()` exposent `CLOCK_MONOTONIC` (durées) et `CLOCK_REALTIME` (timestamps), avec `babet.sleep()` qui respecte l'interruption par signal.

API

Fonction	Renvoie
<code>babet.monotonic()</code>	<code>number</code> — secondes depuis une epoch arbitraire (immune aux changements d'horloge)
<code>babet.now()</code>	<code>number</code> — temps POSIX (secondes depuis 1970-01-01 UTC)
<code>babet.sleep(amount, unit?)</code>	<code>(true, nil) (nil, "interrupted")</code>

`unit` pour `sleep` est l'un de "s" (défaut), "ms", "us". Les floats sont acceptés. (Pas de "ns" : nanosleep ne garantit de toute façon pas cette granularité, et le code ne l'accepte pas.)

API — utilitaires de dates et durées (v1.8.0+)

Ajoutées sous la sous-table `babet.time`. Les trois fonctions plates ci-dessus y sont aussi exposées en alias (`babet.time.now`, `babet.time.monotonic`, `babet.time.sleep`) pour l'ergonomie.

Fonction	Renvoie
<code>babet.time.iso(ts?)</code>	<code>string</code> — "YYYY-MM-DDTHH:MM:SSZ" (UTC). Sans argument, formate l'heure courante.
<code>babet.time.parse_iso(s)</code>	<code>(integer, nil) (nil, "parse_iso: ...")</code> — timestamp Unix en secondes.
<code>babet.time.parse_duration(s)</code>	<code>(integer, nil) (nil, "parse_duration: ...")</code> — nombre de secondes.
<code>babet.time.format_duration(n)</code>	<code>string</code> — style "1d2h3m4s", compact, composantes nulles omises.

Format ISO 8601 accepté par `parse_iso` — pragmatique, avec une règle stricte : **un suffixe de timezone est obligatoire**.

- Séparateur date/heure : T ou espace.
- Timezone : Z (UTC), +HH:MM, ou -HH:MM. **Pas** de +0200, +02, ni d'absence — tous rejetés. Heures 00..23, minutes 00..59.
- Fractions de seconde (.123) acceptées mais ignorées.
- Une string sans timezone est rejetée explicitement plutôt qu'interprétée comme UTC, parce que la plupart des timestamps réels sans Z sont locaux, pas UTC, et la mauvaise interprétation silencieuse est une source de bug classique.

Format de durée accepté par `parse_duration` :

- Unités : s, m (minute), h, d. Pas de w, mo, y, ms, us, ns en v1.
- Composition sans espace : "1h30m", "2d12h", "45m30s", "1d1h1m1s".
- Unités en ordre **strictement décroissant** (d > h > m > s). "30m1h" est rejeté.
- Pas d'unité dupliquée : "1h1h" est rejeté.
- Pas de signe, pas d'espace. String vide rejetée. Un nombre nu sans unité ("5") est rejeté.
- "0s" est la seule représentation canonique de zéro (matche `format_duration(0)`).

Invariant aller-retour : pour tout entier `n >= 0`, `parse_duration(format_duration(n)) == n`. Vérifié dans la suite de tests sur un échantillon représentatif.

Exemple rapide

```
-- Mesurer une durée en toute sécurité (immune aux sauts NTP / DST)
local t0 = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - t0
print(string.format("durée %.3f s", elapsed))

-- Timestamper un événement
local now = babet.now()
db:exec("INSERT INTO events (ts, kind) VALUES (?, ?)", { now, "ping" })

-- Dormir 100 ms, mais se réveiller sur un signal
local ok, err = babet.sleep(100, "ms")
if not ok and err == "interrupted" then
    print("réveillé par un signal")
end

-- Timestamp ISO 8601 pour les logs (toujours UTC)
print("event at " .. babet.time.iso())          -- "2026-06-17T14:32:01Z"
print(babet.time.iso(0))                       -- "1970-01-01T00:00:00Z"

-- Parser une ligne de log en timestamp Unix
local ts = assert(babet.time.parse_iso("2026-06-17T10:00:00+02:00"))
-- ts vaut 1781683200 (qui est 2026-06-17T08:00:00Z)

-- Durées lisibles par un humain
local cache_ttl = babet.time.parse_duration("2h30m") -- 9000
print("cache valide pendant " .. babet.time.format_duration(cache_ttl))
-- "cache valide pendant 2h30m"
```

Contrat d’erreur

- `monotonic()` / `now()` : ne peuvent pas échouer. Renvoient toujours un number.
- `sleep(amount, unit)` :
 - (true, nil) si le sleep s’est terminé normalement.
 - (nil, "interrupted") si un signal géré est arrivé pendant le sleep (voir [signal](#)).
- NaN / Inf / amount négatif → lève via `luaL_error`.
- String d’unit inconnue → lève via `luaL_error`.
- `iso(ts)` hors bornes `int64` (`|ts|` au-delà de ce que le calendrier peut représenter) → lève via `luaL_error` ("iso: value out of int64 range").

Décisions de design

- **Deux horloges, deux fonctions, pas de flag.** `monotonic()` pour les durées, `now()` pour les timestamps. Les fondre en une fonction avec paramètre inviterait au mauvais choix.
- **`now()` pour le temps réel**, pas `realtime()` — plus court, et le sens naturel en anglais matche ce qu’on attend (l’heure actuelle).
- **sleep conscient des signaux.** Un sleep de 60 secondes qui ignore `SIGTERM` est le bug classique de l’arrêt gracieux. `sleep()` renvoie (nil, "interrupted") pour que l’appelant puisse sortir proprement.
- **Pas de helper “format date”.** `os.date(format, babet.now())` marche très bien, et une fonction dupliquée ne serait qu’un wrapper.

Hors v1

- `babet.boottime()` (`CLOCK_BOOTTIME` — inclut le suspend). Rarement utile, facile à ajouter plus tard.

- Helpers `babet.utcnw()` / `localnow()` qui renvoient des strings pré-formatées. Trivial en Lua, sans valeur ajoutée.

babet.socket TLS — connect_tls et starttls

La moitié TLS de `socket` : TCP chiffré pour IRC over TLS, IMAPS, protocoles sécurisés personnalisés. Deux points d'entrée : `connect_tls` pour les protocoles qui font TLS dès le départ (port 6697 IRC, 993 IMAPS, etc.) et `starttls` pour les protocoles qui upgradent une connexion en clair existante (SMTP, IMAP, IRC STARTTLS).

Pourquoi

Parler du TCP chiffré sans `http` est un vrai besoin : bots IRC sur +6697, soumission SMTP, services internes custom derrière TLS. Le faire manuellement avec `openssl s_client` via `exec` n'est pas fiable ; la bibliothèque C++ `openssl` + les abstractions choisies de `cpp-http-lib` donnent une API Lua propre qui gère la vérification de certs correctement par défaut.

API

Fonction	Renvoie
<code>babet.socket.connect_tls(host, port, opts?)</code> <code>s:starttls(opts?)</code>	<code>tls_socket</code> <code>(nil, err)</code> <code>(true, nil)</code> <code>(nil, err)</code> — upgrade un socket en clair existant

`opts` :

Champ	Type	Défaut
<code>verify</code>	boolean — vérification du certificat	<code>true</code>
<code>ca_cert</code>	string (chemin d'un fichier CA bundle)	trust store système
<code>ca_path</code>	string (chemin d'un dossier de CA)	trust store système
<code>hostname</code>	string — SNI + vérification du nom	<code>connect_tls</code> : l'argument <code>host</code> . <code>starttls</code> : requis si <code>verify = true</code>
<code>timeout</code>	number (secondes)	infini — budget appliqué à chaque phase (connexion TCP, puis handshake TLS), pas une enveloppe unique ; la résolution DNS n'est pas couverte

Une ancienne version de cette page nommait l'option `server_name` (le vrai nom est `hostname`) et documentait une option `alpn` qui n'existe pas — voir « Hors v1 ».

Un `tls_socket` a les mêmes méthodes qu'un socket en clair (`send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `close`, `peer`). Le chiffrement est transparent.

Exemples rapides

Connexion à IRC over TLS

```
local s = assert(babet.socket.connect_tls("irc.libera.chat", 6697, {
    timeout = 30,
}))
```

```
s:send("NICK babet-bot\r\n")
s:send("USER babet 0 * :babet bot\r\n")
```

```

while true do
    local line, err = s:recv_line()
    if not line then break end
    print(line)
    if line:match("^PING (.+)$") then
        s:send("PONG " .. line:match("^PING (.+)$") .. "\r\n")
    end
end
end

```

Upgrade STARTTLS (soumission SMTP)

```

local s = assert(babet.socket.connect("mail.example.com", 587))
print(s:recv_line()) -- bannière 220
s:send("EHLO myhost\r\n")
repeat
    line = s:recv_line()
until not line:match("^250%-") -- dernière ligne 250

s:send("STARTTLS\r\n")
print(s:recv_line()) -- 220 ready to start TLS

assert(s:starttls({ hostname = "mail.example.com" }))
-- s est maintenant chiffré ; continue avec EHLO + AUTH + ...

```

Serveur auto-signé (dev / CA privée)

```

local s = assert(babet.socket.connect_tls("internal.svc", 5555, {
    ca_cert = "/etc/myapp/internal-ca.crt",
}))

```

Skip de la vérification (TESTS UNIQUEMENT)

```

local s = assert(babet.socket.connect_tls("self-signed.local", 8443, {
    verify = false,
}))

```

Contrat d'erreur

- Toutes les erreurs préfixées "tls: ..." (handshake, cert verify, etc.) ou "socket: ..." (DNS, connect, timeout).
- (nil, err) toujours — pas d'exceptions pour les échecs "attendus" comme un cert mismatch.
- **verify=true + cert invalide** → (nil, "tls: certificate verify failed: <reason>"). Raisons courantes : expiré, auto-signé, hostname mismatch, CA inconnue.
- **starttls avec verify = true sans opts.hostname** → (nil, "tls: starttls with verify=true requires opts.hostname; pass hostname or set verify=false") — impossible de vérifier un certificat sans savoir quel nom vérifier ; le refus explicite vaut mieux qu'une vérification silencieusement incomplète.
- **Après un échec de starttls** : le socket est rendu **tel quel** — en clair, bloquant, toujours ouvert (tu peux fermer proprement ou réessayer). Cas limite : si la remise en mode bloquant échoue (quasi impossible), le socket est **fermé** et l'erreur porte le suffixe "(socket closed: could not restore blocking mode)" — un état sûr et explicite plutôt qu'un socket au comportement altéré.
- **Timeouts NaN/Inf**, mauvais types → lève via `luaL_error`.

Trust store

Babet embarque sa propre OpenSSL statiquement liée. D'emblée, `verify=true` fonctionne sur :

- **Arch, Debian, Ubuntu, Alpine, Gentoo** : via `--openssldir=/etc/ssl` baké dans l'OpenSSL embarquée.
- **Fedora, RHEL, CentOS, Rocky, Alma, OpenSUSE, FreeBSD, NetBSD** via sondage runtime des chemins de CA bundles connus.

Overrides, par ordre de priorité :

1. `ca_cert` / `ca_path` par appel.
2. Variables d'env `SSL_CERT_FILE` / `SSL_CERT_DIR`.
3. Les deux mécanismes ci-dessus.

Si aucun ne trouve de CA store et que tu utilises `verify=true`, le handshake échoue avec une erreur claire. Voir [security](#) pour le tableau complet.

Décisions de design

- **TLS 1.2 minimum.** SSL 3, TLS 1.0, TLS 1.1 sont inconditionnellement refusés — ils sont cassés, aucun serveur moderne ne devrait s'y reposer. TLS 1.3 est négocié automatiquement quand disponible.
- **`verify=true` par défaut, `verify=false` exige un opt-in explicite.** Aucun moyen de désactiver accidentellement la vérification de cert.
- **hostname défaut sur l'argument `host`** (pour `connect_tls`), pour que SNI marche tout seul. Passe-le explicitement si tu te connectes par IP à un serveur TLS qui utilise SNI — et il est requis pour `starttls` avec vérification, puisque le socket existant ne connaît pas le nom attendu.
- **Mêmes méthodes que socket en clair**, donc une fonction qui prend un `socket` fonctionne avec les deux de manière transparente.

Hors v1

- Négociation ALPN (une ancienne version de cette page documentait une option `alpn` qui n'a jamais existé).
- Authentification client par certificat. Possible à ajouter plus tard en option `client_cert` / `client_key`.
- TLS session resumption / session tickets. Pas courant au niveau script.
- Vérification OCSP stapling. Reposition sur les défauts d'OpenSSL.

babet.toml — parseur TOML

Basé sur [toml++](#), un parser TOML 1.0 strict. Décodage seulement — encoder du TOML fidèlement depuis une table Lua est ambigu (Lua ne distingue pas les inline tables et les dotted tables, par exemple).

Pourquoi

TOML est le format de config naturel des outils modernes (Cargo, pyproject, systemd-credentials, etc.). Un script Babet lisant sa propre config devrait pouvoir le faire directement, pas via une conversion TOML → JSON via `tomlq`.

API

Fonction	Renvoie
<code>babet.toml.decode(text)</code>	table (TOML parsé) (nil, err)

La table renvoyée reflète la structure TOML :

- Strings → strings Lua (UTF-8)
- Integers → integers Lua
- Floats → numbers Lua
- Booleans → booleans Lua
- Datetimes → strings Lua au format ISO 8601 (TOML n'a pas d'équivalent natif Lua ; utilise `os.date` ou une bibliothèque date si tu as besoin des composants parsés)
- Arrays → tables Lua (1-indexées)
- Tables (`[section]` ou `{ inline }`) → tables Lua (avec clés strings)

Exemple rapide

```
local body = [[
title = "My App"
[server]
host = "localhost"
port = 8080
features = ["auth", "metrics"]

[database]
url = "sqlite:///var/lib/app.db"
]]

local cfg, err = babet.toml.decode(body)
if not cfg then error("parsing config échoué : " .. err) end

print(cfg.title)           -- "My App"
print(cfg.server.host)     -- "localhost"
print(cfg.server.features[1]) -- "auth"
print(cfg.database.url)    -- "sqlite:///var/lib/app.db"
```

Contrat d'erreur

- **decode** : (nil, err) en cas d'erreur de parse. Le message d'erreur inclut la ligne/colonne où le parse a échoué, et une courte description de ce qui n'allait pas (par toml++).
- **Argument non-string** → lève via `luaL_error`.

Décisions de design

- **Décode seulement, pas d'encode**. Round-tripper un fichier TOML via une table Lua perd la mise en forme (commentaires, dotted vs inline tables, newlines entre sections) que les vrais utilisateurs apprécient. Si tu dois écrire du TOML, construis la string toi-même — TOML est conçu pour être écrit à la main. Un encodeur fidèle serait un gros chantier pour une valeur limitée.
- **Datetimes en strings**. Le type datetime TOML est riche (offset, local, date-only, time-only) mais Lua n'a pas d'équivalent natif. Renvoyer une string ISO 8601 préserve la donnée sans perte et laisse le script la parser avec les outils de son choix.

Hors v1

- Encodeur. Pourrait être ajouté si un vrai cas d'usage apparaît, mais peu probable.
- Un validateur de schéma. Construis-en un en Lua si nécessaire ; la structure décodée n'est qu'une table.

babet.user

Lookups d'utilisateurs système via NSS, sans parser `/etc/passwd` directement.

Pourquoi

Sur un système Linux moderne, les utilisateurs peuvent venir de plusieurs sources : `/etc/passwd`, LDAP, SSSD, NIS+, FreeIPA, systemd-userdb, ou d'autres plugins NSS. Lire `/etc/passwd` directement rate tout sauf la première source. `babet.user` s'appuie sur `getpwnam_r(3)` et `getpwuid_r(3)`, qui traversent la chaîne de résolveurs configurée dans `/etc/nsswitch.conf` — exactement comme `id` ou `getent passwd` le font.

API

Fonction	Renvoie
<code>user.get(name_or_uid)</code>	<code>table (nil, "user not found") (nil, "user: <err>")</code>
<code>user.exists(name_or_uid)</code>	<code>boolean</code>

`name_or_uid` accepte :

- une **string** → lookup par nom via `getpwnam_r`
- un **integer non-négatif** → lookup par UID via `getpwuid_r`

Tout autre type, un float, un integer négatif, une string contenant un NUL embarqué, ou un integer supérieur à `uid_t` max lèvent via `luaL_error` — ce sont des bugs côté appelant, pas des conditions runtime à gérer proprement.

Table renvoyée en cas de succès

```
{
  name  = "yaourt",
  uid   = 968,
  gid   = 968,
  gecos = "yaourt AUR build user",
  home  = "/var/cache/yaourt",
  shell = "/usr/sbin/nologin",
}
```

Tous les champs string sont garantis non-nil. Ils peuvent être des strings vides quand la source NSS sous-jacente n’a pas de valeur (c’est rare mais possible — typiquement un `gecos` vide).

Exemple rapide

```
-- S'assurer qu'un user système est provisionné avant de lancer
-- le daemon.
if not babet.user.exists("yaourt") then
  error("user yaourt absent – fais d'abord useradd")
end

local u = assert(babet.user.get("yaourt"))
babet.chdir(u.home)
```

Contrat d’erreur

- **Mauvais type d’argument** (table, boolean, nil, float, integer négatif, UID au-dessus de `uid_t` max, string avec NUL embarqué) → lève via `luaL_error`. Utilise `pcall` si tu dois récupérer.
- **Utilisateur absent** (le résolveur ne renvoie aucun match) → `get()` renvoie `(nil, "user not found")` ; `exists()` renvoie `false`.
- **Erreur NSS** (le résolveur lui-même est cassé : LDAP injoignable, mémoire saturée, etc.) → `get()` renvoie `(nil, "user: <description>")` ; `exists()` renvoie `false` (les erreurs sont silencieusement assimilées à “absent” pour les prédicats ; utilise `get()` si tu dois diagnostiquer).

Décisions de design

- **NSS uniquement, jamais `/etc/passwd`** : lire le fichier directement raterait silencieusement tous les comptes qui ne sont pas dans le fichier local. Tout l’intérêt de `babet.user` est que le script obtienne la même réponse que `id` ou `getent passwd`.
- **Variantes `_r` thread-safe** : Babet expose `babet.workers`, donc on utilise `getpwnam_r/getpwuid_r` partout.
- **gecos exposé brut** : le format historique est `Full Name,Office,WorkPhone,HomePhone[,Other]`, mais en pratique la plupart des entrées contiennent juste un nom libre. Parser ça en Lua est trivial si nécessaire ; baker un parser en C++ serait prématuré.

- **Rejet du NUL embarqué** : "root\0evil" serait vu comme "root" par `getpwnam_r` (qui s'arrête au premier NUL). Sur de l'input dérivé d'un utilisateur, ça pourrait contourner une vérification d'identité en amont. Le rejet explicite est plus sûr que la troncature implicite.
- **Check du range uid_t** : sur Linux, `uid_t` est `uint32_t`, alors que `lua_Integer` est `int64_t`. Sans check, `5_000_000_000` serait silencieusement tronqué à 32 bits et pourrait matcher un compte non lié.

Hors v1

Additif — ces choses pourraient être ajoutées plus tard sans casser le SemVer :

- Lookups de groupes (`getgrnam` / `getgrgid`) — sera ajouté quand un cas concret le demandera.
- Accès à `/etc/shadow` (mots de passe, expiration) — hors scope. Exige root et est rarement utile hors scripts admin de niche. L'authentification mot de passe devrait passer par PAM, pas par Babet.
- Création d'utilisateurs/groupe — déjà faisable via `babet.exec("useradd ...")` et `groupadd`. Pas besoin d'un binding dédié.

babet.workers — jobs parallèles

Exécute du code Lua sur de vrais threads OS. Utile pour le travail I/O-bound qui bénéficie de la concurrence (fetcher beaucoup d'URLs, scanner beaucoup de fichiers, traiter des batches) et le travail CPU-bound sur des machines multi-cœurs.

Pourquoi

Lua a des coroutines, mais elles tournent coopérativement sur un seul thread OS — très bien pour les machines à états, pas pour utiliser plusieurs cœurs. Et beaucoup de tâches sont I/O-bound : 100 requêtes HTTP qui prennent 500 ms chacune mettent quand même 50 s séquentiellement, mais ~500 ms en parallèle.

`babet.workers` spawn de vrais threads OS, chacun faisant tourner son propre état Lua isolé, communiquant avec le parent via les arguments passés et les valeurs de retour. Pas d'état partagé signifie pas de locks, pas de data races — juste envoyer du travail, récupérer des résultats.

API

Fonction	Renvoie
<code>babet.workers.spawn(code, args?, opts?)</code>	<code>job (userdata) (nil, err)</code>
<code>job:join()</code>	<code>(true, result) (false, err)</code> — bloquant , voir piège ci-dessous
<code>job:poll()</code>	<code>string</code> — "running" "done" "error", non-bloquant
<code>job:send(v, timeout?)</code>	<code>(true) (false, err)</code> — vers l'inbox du worker
<code>job:recv(timeout?)</code>	<code>(true, v) (false, err)</code> — depuis l'outbox du worker
<code>job:close()</code>	ferme les deux queues ; débloque worker et parent

`opts` : `inbox_capacity` / `outbox_capacity` (entiers, défaut 64) — tailles des files de messages.

`timeout` (secondes) pour `send/recv` : omis ou `nil` = blocage indéfini ; `0` = non-bloquant (`err == "full" / "empty"`) ; sinon attente bornée (`err == "timeout"`). Une queue fermée rend `err == "closed"`. Côté worker, `worker.send(v, timeout?)` et `worker.recv(timeout?)` sont symétriques.

Piège `join()` : interblocage possible. `join()` attend la fin du worker sans échéance. Si le worker est bloqué dans un `worker.recv()` sans `timeout` (il attend un message qui ne viendra jamais), `join()` ne rend jamais la main — le parent et le worker s'attendent mutuellement. Contrairement au ramasse-miettes (qui ferme les queues avant de joindre), `join()` ne ferme rien. Règles pratiques : appelle `job:close()` avant `join()` quand le worker peut être en attente de message ; ou utilise des `timeouts` finis dans les `worker.recv()` ; et `job:poll()` pour tester sans bloquer.

Argument code

Une string source Lua qui tourne dans le worker. Le namespace complet **babet** est disponible, plus une globale **worker** avec :

- **worker.args** — le deuxième argument passé à **spawn**.
- **worker.send(v, timeout?)** / **worker.recv(timeout?)** — le canal de messages vers/depuis le parent (mêmes conventions de timeout que côté parent, cf. table ci-dessus).

La valeur de la dernière expression du worker (ou **return value**) est ce que **join()** renvoie comme **result**.

Argument args

Toute valeur qui peut être deep-copiée entre états Lua : **nil**, **booleans**, **numbers**, **strings**, **tables** des mêmes. **Pas** : **fonctions**, **userdata**, **threads**, **tables** avec cycles. Les tables sont deep-copiées — pas de partage. Profondeur maximale : **32 niveaux** d'imbrication (au-delà : (**nil**, "workers: spawn: value too deeply nested")). Mêmes règles pour la valeur de retour du worker et pour les messages **send/recv**.

Exemples rapides

Fetches HTTP parallèles

```
local urls = {
    "https://a.example/",
    "https://b.example/",
    "https://c.example/",
    "https://d.example/",
}

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { ok = false, err = err } end
        return { ok = true, status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    if ok then
        print(i, result.status or result.err)
    else
        print(i, "worker crashed :", result)
    end
end
```

Travail CPU-bound avec cancellation

La cancellation est **coopérative**, construite avec le canal de messages : le parent envoie "stop", le worker vérifie son inbox périodiquement avec **worker.recv(0)** (non bloquant).

```
local jobs = {}
for i = 1, 4 do
    jobs[i] = babet.workers.spawn([[
        local chunk = worker.args.chunk
        local total = 0
        for x = chunk.from, chunk.to do
            if x % 1000 == 0 then
```

```

        local got, msg = worker.recv(0) -- non bloquant
        if got and msg == "stop" then
            return { cancelled = true, partial = total }
        end
    end
    total = total + heavy_compute(x)
end
return { total = total }
]], { chunk = { from = i * 1000, to = (i + 1) * 1000 - 1 } })
end

-- ... plus tard, si nécessaire :
-- for _, j in ipairs(jobs) do j:send("stop") end
-- puis join() pour récupérer les résultats partiels.

```

Pattern pool de workers

```

local function map_parallel(items, code, max_concurrent)
    -- pas de cpu_count() en v1 : fixe une valeur, ou détecte :
    -- tonumber(babet.exec("nproc").stdout)
    max_concurrent = max_concurrent or 4
    local results = {}
    local i = 1
    local active = {}

    while i <= #items or next(active) do
        -- en lancer autant que possible
        while i <= #items and #active < max_concurrent do
            active[#active + 1] = {
                index = i,
                job = babet.workers.spawn(code, { item = items[i] }),
            }
            i = i + 1
        end
        -- moissonner les finis
        for k = #active, 1, -1 do
            local entry = active[k]
            if entry.job:poll() ~= "running" then
                local ok, result = entry.job:join()
                results[entry.index] = ok and result or { err = result }
                table.remove(active, k)
            end
        end
        if next(active) then babet.sleep(10, "ms") end
    end
    return results
end

```

Contrat d'erreur

- **spawn** : (nil, err) si le thread OS ne peut pas être créé (rare — généralement des limites de ressources).
- **join** :
 - (true, result) — worker terminé normalement ; result est sa valeur de retour (ou nil s'il n'a pas retourné).
 - (false, err) — worker crashé avec une erreur non attrapée ; err est le message d'erreur + traceback.
 - (false, "workers: join: result already consumed") — join déjà appelé sur ce job.
 - join() n'a **pas** de timeout : il bloque jusqu'à la fin du worker (cf. le piège d'interblocage plus haut).

- **Mauvais types d'argument** → lève via `luaL_error`.

Partage de données

Les workers ne partagent pas l'état Lua. La communication se fait :

- **En entrée** : via l'argument `args` de `spawn` (deep-copié dans l'état du worker).
- **En sortie** : via la valeur de retour du worker (deep-copiée en retour).
- **Effets de bord** : un worker peut écrire dans un fichier, appender dans un log, requêter une base de données — mais la coordination via ces effets de bord est la responsabilité du script.

Si deux workers écrivent dans le même fichier SQLite, positionne `wal=true` à `open` pour qu'ils ne se verrouillent pas mutuellement. SQLite gère la synchronisation correctement avec WAL ; dans `busy_timeout` tu peux configurer combien de temps attendre sur une db occupée.

Décisions de design

- **Un état Lua par worker, pas de partage**. L'alternative — état partagé avec locks — est une source bien connue de bugs subtils et on ne pense pas que les locks niveau Lua valent le travail de design à ce niveau. Le message-passing pur est plus simple.
- **Pas de thread pool global**. Chaque `spawn` crée un nouveau thread OS, chaque `join` le ferme. Pour les boucles serrées, construis un pool en Lua (voir l'exemple ci-dessus). Le modèle plus simple gagne en lisibilité.
- **Le premier spawn verrouille `setenv` et `chdir`** — définitivement, même après `join`. L'environnement et le répertoire courant sont des états process-wide ; les muter pendant qu'un worker tourne est une course de données. Fixe-les avant le premier worker. Voir la note dans [sys](#).
- **`babet.signal` n'est pas disponible dans les workers**. Les signaux sont process-wide ; seul le thread principal peut les gérer sensément.

Hors v1

Note (audit v21) : les éléments ci-dessous relèvent du design initial et ne sont **pas implémentés à ce jour** — l'ancienne version de cette page les présentait à tort comme disponibles : `job:join(timeout)`, `job:done()` (utiliser `job:poll()`), `job:cancel()` / `worker.cancelled()` (une cancellation coopérative peut se construire dès aujourd'hui avec `job:send("stop")` + un `worker.recv(0)` périodique), et `babet.workers.cpu_count()`.

- Channels **entre workers** (worker worker — le canal parent worker existe déjà : `send/recv`). À ajouter plus tard si un cas d'usage montre que le pattern est courant.
- Mémoire partagée (mmap). Idem.
- Futures I/O async (style `spawn().then(...)`). Hors scope ; utilise une boucle de polling avec `job:poll()`.

Cookbook

Recettes concrètes pour des tâches de scripting courantes. Chaque recette est courte et autonome ; copie, adapte, déploie.

Surveiller un dossier et envoyer les nouveaux fichiers par email

Un dossier de dépôt qui mailise chaque fichier dès qu'il arrive. Utilise `babet.inotify` pour le réveil instantané (pas de polling), et écoute `close_write` + `moved_to` pour garantir que le fichier est complet.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)
```

```

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue saturée : rescane le dossier
            -- (des événements ont été perdus)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
end

```

S'assurer qu'un user système existe, puis basculer dessus

Check typique à l'install avant de lancer un daemon sous un utilisateur non-root.

```

local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd a échoué : " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)

```

Récupérer beaucoup d'URLs en parallèle

babet.workers fait tourner du Lua sur plusieurs cœurs. Le temps total est proche du fetch le plus lent, pas de la somme.

```

local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

```

```

end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end

```

Arrêt propre d'un script long-running

Attraper SIGTERM / SIGINT pour que le script puisse fermer ses ressources proprement. Fonctionne avec `systemctl stop` et Ctrl-C.

```

local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, arrêt en cours")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, arrêt en cours")
    running = false
end)

while running do
    -- boucle principale ; les appels bloquants renvoient
    -- (nil, "interrupted") quand un signal géré arrive, donc la
    -- boucle sort rapidement.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv : ", err); break
    end
    process(line)
end

sock:close()
log.info("arrêt propre")

```

Lire une config TOML, valider, persister en SQLite

```

-- Charger le TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml : " .. perr) end

-- Valider (checks manuels simples ; pour un vrai schéma, bâtis-le en Lua)
assert(type(cfg.server) == "table", "section [server] absente")
assert(type(cfg.server.host) == "string", "server.host absent")
assert(math.type(cfg.server.port) == "integer", "server.port doit être un entier")

-- Ouvrir la db, persister
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",

```

```

        { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "port", tostring(cfg.server.port) })
db:close()

```

Cache avec TTL lisible, échéances en timestamps ISO

Parser un TTL depuis une config ("15m", "2h30m", "1d"), calculer une échéance, et la logger dans un format UTC uniforme sur lequel `grep` à travers les fichiers de log marche vraiment.

```

-- Config depuis TOML, CLI, env, peu importe.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("TTL invalide '" .. raw_ttl .. "' : " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] entrée cache créée, expire à %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...plus tard, ailleurs dans le script :
if babet.now() >= deadline then
    print "[" .. babet.time.iso() .. "] cache miss : entrée expirée"
end

```

Propriétés utiles de ce pattern :

- `babet.time.iso()` est toujours UTC, toujours 20 caractères, toujours la même forme. `grep` et `sort` marchent directement sur les timestamps.
- `parse_duration("15m")` renvoie un entier, donc `now() + ttl` est exact (pas de drift float sur les longues durées).
- `format_duration(ttl)` est aller-retour : `parse_duration( format_duration(n) ) == n` pour tout `n >= 0`. Sûr à utiliser comme représentation canonique dans les logs ou les bases.

Pretty-print d'une valeur Lua

Le module `inspect` (embarqué) donne une sortie lisible pour les tables imbriquées.

```

local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }

```

Hello there

Le script Babet minimum :

```

babet.helloThere()

```